

Control flow statements are coding mechanisms developers use to break up execution flow of a program by employing *decision making*, *looping*, and *branching* constructs to enable programs conditionally execute particular blocks of code.

This unit discusses the decision-making statements - *if-then*, *if-then-else*, *switch*, the looping statements - *for*, *while*, *do-while*, and the branching statements - *break*, *continue*, *return* supported by the Java programming language.

DECISION MAKING

(The if-then, if-then-else and switch Controls Statements)

The if-then Statement

The syntax for the *if-then* block is:

```
if (conditional statement) {  
    //the then clause: codes to execute if condition is true  
}
```

When the Java compiler during program execution gets to this point, it pauses and evaluates the conditional statement in the if-then code block. If the conditional statement evaluates to true, the then clause is executed, otherwise the execution skips the then clause to proceed to execute any code beyond the if-then block. The if-then is called a block because, the structure envelopes a group of Java statements within an opening brace `{}` and a corresponding closing brace `}`.

Applying the syntax to our example, one possible pseudocode⁴ implementation of the investment decision could be as follows:

```
if (isAbove 15%) {  
    //the then clause: invest in the money market  
}
```

The opening and closing braces are optional, provided that the "then" clause contains only **one** statement, thus we could alter our investment code to:

```
if (isAbove 15%)  
    invest
```

Deciding when to omit the braces is a matter of personal choice. Omitting them could make the code difficult to understand. If a second statement is later added to the "then" clause, a common mistake could be forgetting to add the required braces. The compiler cannot catch this sort of error; you'll just get the wrong results.

2. Type the following code into your class

```
1. package ifDecision;
2. public class Numbers {
3.     public static void main(String[] args) {
4.         for(int i=1; i<=3; i++)
5.         {
6.             for(int j=1; j<=3; j++)
7.             {
8.                 System.out.println(i+j);
9.             }
10.        }
11.    }
12. }
13. }
```

3. Run your program

4. Your output should be similar to figure 2-3.5

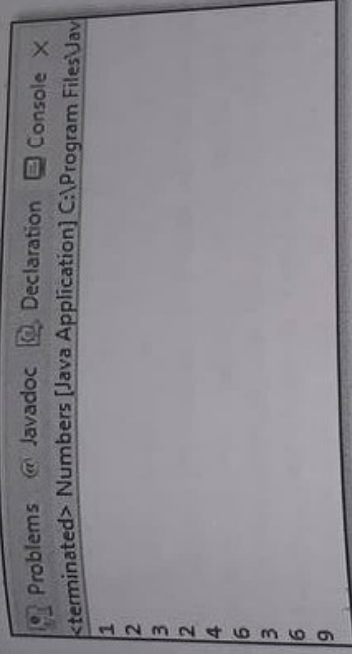


Figure 2-3.5 Numbers class output

The program uses nested for statements; the first iteration of the outer loop prints 1, 2, and 3. For the second iteration of the outer loop, it prints 2, 4, and 6 and for the third iteration of the outer loop, it prints 3, 6, and 9.

Second code:

We shall repeat the example in the first code but, we shall add the continue label to the code. Follow the steps below to complete the task.

1. Create another class in decision project; name the class *LabelledNumbers*
2. Type the following code into your class

```
1. package ifDecision;
2.
3. public class LabelledNumbers {
4.     public static void main(String[] args) {
5.         outer_loop:
6.         for(int i=1; i<=3; i++)
7.         {
```

The if-then-else Statement

You could apply the if-then-else block in the investment example to take some action if the returns on investment is less than 15%. In this case the action is simply not to invest; coded as follows:

```
if (isAbove 15%) {  
    //the true part: invest  
} else {  
    //the false part: do no invest  
}
```

If-then-Else Example 1

In this example, we will create a program that assigns a grade based on the value of a test score: An A for a score of 90% or above, a B for a score of 80% or above, and so on. Follow the steps below to complete the example.

1. Create a new project in the Package Explorer. Click File >> New >> Java Project.
2. Name the project *decision*. Click Finish
3. Name the module *unit2.decision*. Click Create
4. Create a package to organize our codes. Right-click the *src* folder in the project. Select New
5. Select Package. Name the Package *ifDecision*. Click Finish
6. Create a class in the package. Right-click the *ifDecision* package >> select New >> select Class
7. Name the class *TestGrade*
8. Code the class as shown in figure 2-1.1

```
TestGrade.java X  
1 package ifDecision;  
2  
3 import java.util.Scanner;  
4  
5 public class TestGrade {  
6     public static void main(String[] args) {  
7  
8         Scanner scanner = new Scanner(System.in);  
9         float testScore;  
10        char grade;  
11  
12        System.out.println("Enter your test score");  
13        testScore = scanner.nextFloat();  
14  
15        if (testScore >= 90) {  
16            grade = 'A';  
17        } else if (testScore >= 80) {  
18            grade = 'B';  
19        } else if (testScore >= 70) {  
20            grade = 'C';  
21        } else if (testScore >= 60) {  
22            grade = 'D';  
23        } else {  
24            grade = 'F';  
25        }  
26        System.out.println("Grade = " + grade);  
27    }  
28 }
```



```

8.         inner_loop:
9.         for(int j=1; j<=3; j++)
10.         {
11.             if(i*j == 3)
12.                 continue outer_loop;
13.             System.out.println(i*j);
14.         }
15.     }
16. }
17. }

```

3. Run your program
4. Your output should be similar to that shown in figure 2-3.6

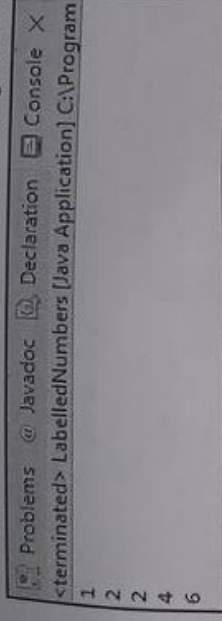


Figure 2-3.6 Labeled numbers class output

The second program uses a *continue* statement with the outer loop whenever the condition ($i*j = 3$) is satisfied. For the first iteration of the outer loop, we get ($i*j$) as 1, 2 and 3. 3 is not printed as the statements after the *continue* statement are not executed for the first iteration, then execution control moves to the second iteration of the outer loop. Since we never get ($i*j$) as 3 in the second iteration, 2, 4 and 6 are printed. In the third iteration of the outer loop, we get ($i*j$) as 3, 6 and 9. Since the first value is 3, the *continue* statement is executed and the control moves to the next iteration and none of the values are printed in this iteration. As there is no next iteration, the loop exits.

3The return Statement

The last of the branching statements is the return statement. The return statement exits a current method, and flow control returns to where the method was invoked. The return statement has two forms: one that returns a value, and one that doesn't. To return a value, simply put the value (expression) after the return keyword.

The syntaxes for the return statement are:

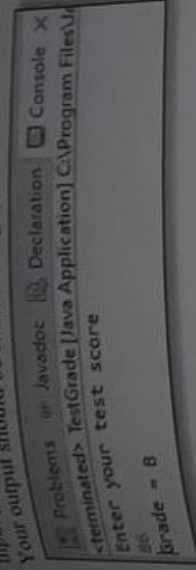
- (A) Syntax of return statement that returns a value:
return value;
- (B) Syntax of return statement that does not return a value:
return;

The data type of a returned value must match the return data type of the method that declares it. When a method is declared *void*, it means that method does not return a value thus, uses the syntax B above. We will cover more on methods later in the book.

9. Run your application. Right-click the *TestScore.java* file in the Package Explorer >> select Run As >> select Java Application.

10. Input your test score and press the Enter key

11. Your output should be similar to figure 2-1.2



Switch Block Example 1

This example demonstrates how to use the switch block. We will code a program to display the name of the month of the year when a user types the month's number. Follow the steps below to accomplish the task.

1. Create a new class in the decision project; name the class *Month*

2. Type the following code into your class. Do not repeat the class name

```
1. package ifDecision;
2. import java.util.Scanner;
3. public class Month {
4.     Scanner scanner = new Scanner (System.in);
5.     int month;
6.     String nameOfMonth;
7.     System.out.println("Type the corresponding number for the current
8.     month");
9.     month = scanner.nextInt();
10.     switch (month) {
11.         case 1: nameOfMonth = "January";
12.             break;
13.         case 2: nameOfMonth = "February";
14.             break;
15.         case 3: nameOfMonth = "March";
16.             break;
17.         case 4: nameOfMonth = "April";
18.             break;
19.         case 5: nameOfMonth = "May";
20.             break;
21.         case 6: nameOfMonth = "June";
22.             break;
23.         case 7: nameOfMonth = "July";
24.             break;
25.         case 8: nameOfMonth = "August";
26.             break;
27.         case 9: nameOfMonth = "September";
28.             break;
29.         case 10: nameOfMonth = "October";
30.             break;
31.         case 11: nameOfMonth = "November";
```

UNIT 3 THE ARRAY COLLECTION OBJECT

ARRAYS

Structure of Array Object

As said before, an array is a contiguous allocation of memory spaces which stores data items of a single type. Each item in an array is called *element*, each element has a single storage space within an array. Storage spaces in array are identified by numerical *indices*. Index numbers of arrays always begin from 0 to the end of the array. Figure 3-1.1 illustrates array structure:

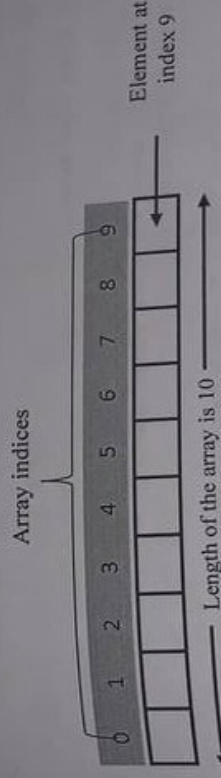


Figure 3-1.1 Array structure

The illustration shows an array structure that stores *ten elements*; as a rule, the elements must be of a single type. Each element is identifiable by an index number. As shown in the illustration, the 10th element is accessible (identifiable) by index 9. The syntax of an array is given by:

- (A) `data type [] nameOfArray;`
- (B) `data type nameOfArray [];`

Any of the two types of declaration is acceptable, however, the preferred type is option (A). Option (A) is the original structure defined by the Java language. Option (B) was created for developers with C programming language background.

Every array knows its length, so to find the length of any array we use the *length* field which is automatically created together with the array. To get the length, use the syntax: *array.length*, where *array* is the name of an already declared array. For example, using the illustration in figure 3-1.1, assuming the array is named *myArray*, we could get its length by coding *myArray.length*. Let's turn the words into code. Let's consider an example in which we will:

- declare an array
- initialize an array and
- print elements in array.

Array Example 1

We will cover two examples, in example 1, we will declare an array of only numerical data values supplied by the developer (hard coding).

*j = 3) is
statements
moves to the
are printed.
are continue
printed in this

method,
: one that
the return

When a
ve. We

The second example will dynamically prompt users for numerical input into the array. Let's get started already; follow the steps to complete the task.

1. Create a new Java project; call the project *Collection* >> click the Finish button
2. Name the module *unit3.collection* >> click the Create button
3. In the src folder in the Collection project, create a package. Right-click the src folder >> select New >> select Package
4. Name the package *arrays*
5. In the *arrays* package, create a class. Right-click the package *arrays* >> select New >> select Class
6. Name the class *FirstArray*
7. Type the code into your class. Remember not to repeat the class name.

```

1. package arrays;
2.
3. public class FirstArray {
4.
5.     public static void main(String[] args) {
6.         // declare an array to store float numbers
7.         float [] myArray;
8.
9.         // indicate number of elements to store
10.        myArray = new float [5];
11.
12.        // initialize array with inputs from developer
13.        myArray [0] = 20.5f;
14.        myArray [1] = 72.8f;
15.        myArray [2] = 20.5f;
16.        myArray [3] = 42.8f;
17.        myArray [4] = 87.2f;
18.
19.        // display the content of the array
20.        System.out.println("Element @ index 0: is " + myArray[0]);
21.        System.out.println("Element @ index 1: is " + myArray[1]);
22.        System.out.println("Element @ index 2: is " + myArray[2]);
23.        System.out.println("Element @ index 3: is " + myArray[3]);
24.        System.out.println("Element @ index 4: is " + myArray[4]);
25.    }
26.
27. }

```

8. Run your program

9. Your output should be similar to figure 3-1.2

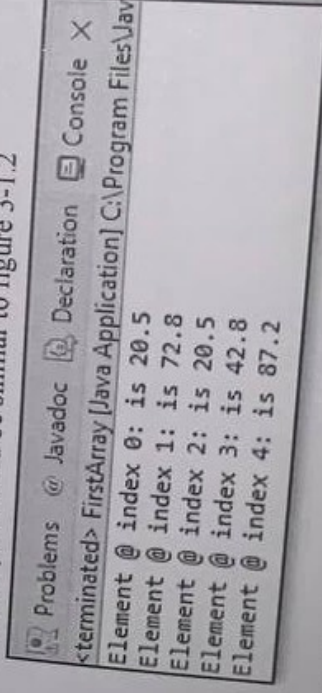


Figure 3-1.2 First array class output

Array Example 2
Let's consider a d
initialized the arr
We will need to
away. follow the

1. Create th
2. Type th
1. package
2. import
3. publi
- 4.
5. //
6. //
- 7.
- 8.
- 9.
10. //
- 11.
- 12.
13. //
- 14.
- 15.
- 16.
- 17.
- 18.
- 19.
20. /
- 21.
- 22.
- 23.
- 24.
- 25.
- 3.
- 4.

der >> Let's get started

w >> Select New

w >> select Class

Array Example 2

We'll consider a different approach of populating an array, in the previous example, the developer let's consider the array. This example will ask a user to initialize the array with a specific number of inputs. We will need to use a looping construct – preferably a for statement for this purpose. Let's get to it right away, we will follow the steps to complete the task.

1. Create a new class in the Collection project; name it *SecondArray*
2. Type the following code into the class

```
1. package arrays;
2. import java.util.Scanner;
3. public class SecondArray {
4.     public static void main(String[] args) {
5.         // library resource to receive user input
6.         Scanner scanner = new Scanner(System.in);
7.         double largest = 0f;
8.
9.         // declare double array, reserving 5 memory spaces to store user
10.        double[] anArray = new double[5];
11.
12.        // accept user input
13.        for (int i = 0; i < anArray.length; i++) {
14.            System.out.println("input a number to store @ array
15.            index:" + i);
16.            anArray[i] = scanner.nextDouble();
17.        }
18.
19.        // display content of the array
20.        for (int i = 0; i < anArray.length; i++) {
21.            System.out.println("Element @ index " + i + " is: " +
22.            anArray[i]);
23.        }
24.    }
25. }
```

3. Run your application
4. Your output should be similar to figure 3-1.3


```
Problems 6r Javadoc Declaration Console X
<terminated> SecondArray [Java Application] C:\Program Files\
7 input a number to store @ array index:0
input a number to store @ array index:1
45
input a number to store @ array index:2
254
input a number to store @ array index:3
56
input a number to store @ array index:4
47
Element @ index 0 is: 7.0
Element @ index 1 is: 45.0
Element @ index 2 is: 254.0
Element @ index 3 is: 56.0
Element @ index 4 is: 47.0
```

Figure 3-1.3 Second Array class output

Array Example 2: Code Explanation

In the code we see a different way to declare an array; line 13 declares the array as this: `double[] anArray = new double[5]` and calls it `anArray`. Unlike the previous example, the number of elements the `anArray` holds is initialized to the array on the same line 12.

Using a `for` statement, we accept all five user inputs; the `for` begins from 0 because the array's index starts from 0. The `for` statement loops one less than the length of the array number of times, meaning the last value of `i` is 4, which is also the last index number of the `anArray` structure. The control variable of the `for` loop is used to also control the current index position of the array. At the beginning of the loop, the value of `i` is 0, the conditional expression `i < anArray.length` is checked, it evaluates to true because `i` which is 0 at this point is less than the length of the array which is 5. The body of the loop gets executed because the evaluation returned true. In the body of the loop, we accept user input into array position `i` with the code `anArray[i] = scanner.nextDouble()`. (Remember, `i` is still zero at this point).

At this point, user's first input has been assigned to the array's index position [0] and since there are no more statements to execute in the body of the loop, flow control moves to the third part of the `for` statement to increment `i` by 1. Now, the current value of `i` is 1; `i` will be evaluated again against the conditional expression. The evaluation will return true because `i`, which is 1, is less than the length of the array which is 5. The body of the loop gets to execute again because the evaluation returned true. The same process happens when `i` is incremented to 2, 3 and 4.

After the body of the loop is executed when `i` is 4, flow control moves to the third part of the loop to increment `i` by once again. Current value of `i`, is now 5; conditional expression is evaluated again and the result is false because, `i` which is now 5 after the increment is not less than length of the array which also is 5. At this point, flow control jumps out of the loop to continue execution from line 23.

A similar execution pattern occurs at the for loop at line 23. The for statement is used to loop through the array positions using i as the control value to print elements in the array.

Once you are able to receive inputs into an array, you could perform several processes on the elements as though they were kept in separate containers. We could sum up all the elements in our array, or sum portions of it, find the largest element or the smallest element, etc. Let's find the largest input in the anArray structure using the following code.

```

1. package arrays;
2. import java.util.Scanner;
3. public class SecondArray {
4.     public static void main(String[] args) {
5.         // library resource to receive user input
6.         Scanner scanner = new Scanner(System.in);
7.         double largest = 0f;
8.
9.         // declare double array, reserving 5 memory spaces to store user
10.        double[] anArray = new double[5];
11.
12.        // accept user input
13.
14.        for (int i = 0; i < anArray.length; i++) {
15.            System.out.println("input a number to store @ array
16.                                index:" + i);
17.            anArray[i] = scanner.nextDouble();
18.        }
19.
20.        // display content of the array
21.        for (int i = 0; i < anArray.length; i++) {
22.            System.out.println("Element @ index " + i + " is: " +
23.                                anArray[i]);
24.        }
25.        // finding the largest element....
26.        // copy the first element and use for comparison
27.        largest = anArray[0];
28.
29.        // loop through array and make comparison
30.        for (int i = 1; i < anArray.length; i++) {
31.            if (anArray[i] > largest) {
32.                largest = anArray[i];
33.            }
34.        }
35.        // display largest input
36.        System.out.println("largest element in the array is: " +
37.                            largest);
38.    }
39. }

```

double[]

ber of elements

array's index starts
 finding the last
 variable of the for
 loop, the value
 because i which
 executed because
 on i with the

there are no
 he for
 inst the
 length of the
 if true. The

the loop to
 again and the
 y which also

MULTI-DIMENSIONAL ARRAY

A multi-dimensional array is an array that has more than one dimension or an array that has multiple levels. The simplest multi-dimensional array is the two-dimensional (2D) array, 2D array is also called a matrix, or a table of rows and columns. A multi-dimensional array is technically an array of arrays, mostly referred to as 3D, 4D, etc. We shall look at how to use the 2D array, 3D, 4D and the rest of it are out of the scope of the book.

Two-dimensional Array (2D) Object

A two-dimensional array essentially is a tabular structure; a grid of rows and columns. Thus, any data stored in a tabular or matrix form could be stored in a 2D array structure. In Java, 2D array is created as an array of arrays, thus a 2D array is declared similar to a 1D array (single-dimensional array) object.

The syntax for a 2D array is given as:

```
data type[][] arrayName;
```

Example:

```
int[][] salesRecord;  
salesRecord = new int[3][4];
```

This creates a *three-by-four* 2D array of int type which can store up to twelve elements arranged in 3 rows and 4 columns.

Accessing a 2D Array Object

Think of a tabular structure each time you want to access elements in a 2D array. To access data in a table, you need to know the row number first, then the column number of the data next. Similarly, the first index of the first array of a 2D array can be thought of as a *row*, the second index of the second array can be seen as the *column* number. Just like 1D array, 2D arrays indices starts from 0.

Array Initializer Lists

Array initializer lists are used in Java to quickly give initial values to a 2D array object. Array initializer for a 2D array is a list which contains rows, separated by commas and enclosed between braces. Within each row is a list of column values separated by commas. You could initialize a 2D array in two ways:

- by using curly brackets

Example:

```
//Optio  
doubl1
```

b. by using th

```
//Optio  
string  
string
```

Modify 2D Elen

Elements in a 2D
array, you need

```
//Op  
stri  
stri  
str  
Sys  
Sys
```

Figure

Figure 3-2.3
“Conduat”

```
<t  
Ar  
C
```

Traversin

“Row-ma
from the
nested lo
through
loop wil

Examp


```
//Option a: declare 2D array and initialize it at the same time
double[][] salesRecords = {{2.4, 3.5, 7.5}, {7.3, 12.5, 13.8},
                           {45.48, 152, 24.3}, {10.3, 14.5, 21.8}};
```

b. by using the *new* keyword

```
//Option b: declare 2D array and initialize it separately;
String[][] stringValues;
stringValues = new String[][] {{ "Abeiku", "French"}, {"Andrew", "Kpobi"},
                               {"Aba Duncan", "Ato Ferguson"}};
```

Modify 2D Element(s)

Elements in a 2D array can be modified or changed just as in a 1D array. To modify elements in a 2D array, you need to specify the row number and column of the element to modify. Example:

```
//Option b: declare 2D array and initialize it separately;
String[][] stringValues;
stringValues = new String[][] {{ "Abeiku", "French"}, {"Andrew", "Kpobi"},
                               {"Aba Duncan", "Ato Ferguson"}};

stringValues[1][1] = "Conduah";

System.out.println(stringValues[1][0]);
System.out.println(stringValues[1][1]);
```

Figure 3-2.3 2D array modification

Figure 3-2.3 show how to modify a 2D array, the code will cause Andrew's surname to be modified to "Conduah" (See figure 3-2.4).

```
Problems @ Javadoc Declaration Console X
<terminated> CopyArray [Java Application] C:\Program Files\Ja
Andrew
Conduah
```

Traversing 2D array - Row-major Order

"Row-major order" refers to an ordering of 2D array elements where traversal occurs across each row; from the top left corner to the bottom right. In Java, row-major ordering can be implemented by using nested loops where the outer loop variable iterates through the rows and the inner loop variable iterates through the columns. Note that inside these loops, when accessing elements, the variable used in the outer loop will be used as the first index, and the inner loop variable will be used as the second index.

Example:

as multiple
is also called a
of arrays,
e rest of it are

, any data
s created as
(V) object.

ed in 3 rows

i in a
y, the first
array can

tializer
Within
ays:

```

for (int i = 0; i < stringValues.length; i++) {
    System.out.print("\t");
    for (int j = 0; j < stringValues[i].length; j++) {
        System.out.print(stringValues[i][j]);
        System.out.print(" ");
    }
}

```

Output:

```

Problems @ Javac Declaration Console X
terminated> CopyArray [Java Application] C:\Program Files\Java\jdk-17.0.1\bin\javaw.exe
Abeiku French Andrew Kpobi Aba Duncan Ato Ferguson

```

Traversing 2D array - Column-major Order

“Column-major order” refers to an ordering of 2D array elements where traversal occurs down each column, from the top left corner to the bottom right. In Java, column major ordering can be implemented by using nested loops where the outer loop variable iterates through the columns and the inner loop variable iterates through the rows. Note that inside these loops, when accessing elements, the variable used in the outer loop will be used as the second index, and the inner loop variable will be used as the first index.

Example:

```

for (int i = 0; i < stringValues[0].length; i++) {
    for (int j = 0; j < stringValues.length; j++) {
        System.out.println(stringValues[j][i]);
    }
}

```

Figure 3-2.7 Column-major order

Output:

```

Problems @ Javac Declaration Console X
terminated> CopyArray [Java Application] C:\Program Files\Java\jdk-17.0.1\bin\javaw.exe
Abeiku
Andrew
Aba Duncan
French
Kpobi
Ato Ferguson

```

Figure 3-2.8 Column-major order output

2D Array Example 1

We shall look at example which creates a program that stores exams marks of six students who study three different courses in their Software Engineering program. Before we commence writing the program, let's picture the 2D structure with the help of table 3-2.1.

Table 3-2.1 Exam score of students

Name of student	Java	VB.Net	Python
Abigail	89	78	75
Prince	74	81	70
Sylvester	69	73	77
Emmanuel	68	70	72
Akua	88	85	82
Afiya	85	91	65

Table 3-2.1 uses six rows and three columns to store the records of the six students, so essentially, it is a six-by-three table excluding the top row and first column. The table could therefore be represented using a 2D array created in this manner:

```
int[] [] testScore = {{89, 78, 75}, {74, 81, 70}, {69, 73, 77},
                      {68, 70, 72}, {88, 85, 82}, {85, 91, 65}};
```

The six rows in table 3-2.1 represents number of students and the three columns represents the number of courses pursued by the students. Critically observing the testScore array initialization list shows that each set of data (row) is a 1D array. Therefore, this 2D array (testScore) contains six 1D arrays and so they could be referred to as an array of arrays or a multi-dimensional array. The following steps creates a program which declares the testScore 2D array, populates it, traverses it and prints the highest exam score in the 2D array.

1. Create a new class in the Collection project; name the class *MultiArray*
2. Type the following code into your class

```
1. package arrays;
2.
3. public class MultiArray {
4.
5.     public static void main(String[] args) {
6.         // Create 2D array
7.         int[][] testScore = {{89, 78, 75}, {74, 81, 70}, {69, 73, 77},
8.                               {68, 70, 72}, {88, 85, 82}, {85, 91, 65}};
9.
10.        // Declare and initialize temp variable
11.        int max = Integer.MIN_VALUE;
12.
13.        for (int row = 0; row < testScore.length; row++) {
14.            for (int col = 0; col < testScore[row].length; col++) {
15.                int value = testScore[row][col];
16.                if (value > max){
17.                    max = value;
```

```

18.         }
19.     }
20.     System.out.println(max);
21. }
22. }
23. }
24. }

```

3. Run your program
4. Your output should be similar to figure 3-2.9

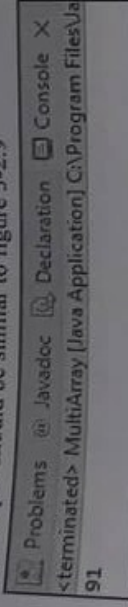


Figure 3-2.9 Multi array class output

2D Array Example 1: Code Explanation

Line 7 creates the `testScore` 2D array and initializes the array with the list in the braces. Line 11 (`Integer.MIN_VALUE`) represents a minimum constant value that an integer can have. Line 13 to 20 is a nested for loop which determines the maximum exams score. The outer for statement loops through the rows, the inner for statement loop through the columns. The code at line 15 `int value = testScore[row][col]`, moves the element at a particular position (which is identifiable using row number and column number) to the variable `value`.

Line 16 checks if the element in the variable `value` is greater than the what the `max` container stores, if the evaluation returns true, then the element in the variable `value` is assigned to the `max` container at line 16. Line 21 merely prints the content of `max`.

2D Array Example 2

In the previous example we saw how a 2D could be initialized with a list, in this example will consider an example which uses the **new** keyword to initialize a 2D array. In the previous example, we could use the initialization list approach because the list to initialize the array with was known beforehand. In this example we will allow a user to indicate the size of the 2D array and also determine the values to initialize the array with. Follow the steps to code the program.

1. Create a class in the Collection project; name the class `NewMultiArray`
2. Type the codes below into your class

```

1. package arrays;
2.
3. import java.util.Scanner;
4.
5. public class NewMultiArray {

```

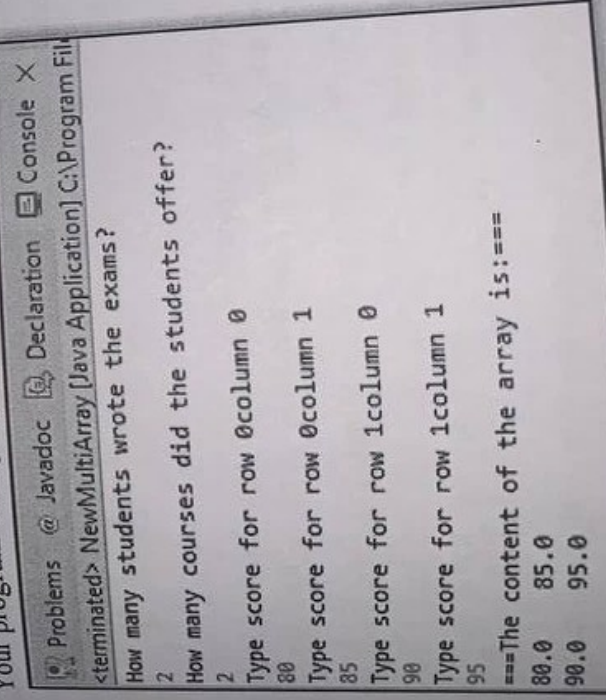


```

6. public static void main(String[] args) {
7.     Scanner sc = new Scanner(System.in);
8.     System.out.println("How many students wrote the exams? ");
9.     int rows = sc.nextInt();
10.    System.out.println("How many courses did the students offer? ");
11.    int columns = sc.nextInt();
12.
13.    // create 2D array, initialize with new keyword
14.    double[][] examScore = new double[rows][columns];
15.
16.    // Take user inputs into 2D array
17.    for (int row = 0; row < examScore.length; row++) {
18.        // for (int col = 0; col < examScore[row].length; col++) {
19.            System.out.println("Type score for row " + row +
20.                               "column " + col);
21.            examScore[row][col] = sc.nextDouble();
22.        }
23.    }
24.
25.    // display exams score
26.    for (int row = 0; row < examScore.length; row++) {
27.        for (int col = 0; col < examScore[row].length; col++) {
28.            System.out.print(examScore[row][col]);
29.            System.out.print("\t");
30.        }
31.        System.out.println();
32.    }
33.
34.    // }
35. }

```

3. Run your program. Supply inputs for the program, press the Enter key after each input
4. Your program's output should be similar to the output in figure 3-2.10



2D Array Example 2: Code Explanation

The initialization of the 2D array in this example occurs when the user indicates to the program the number of students and courses there are. Line 16 initializes the 2D with the values supplied by the user which will be arranged in the 2D array in **rows** and **columns**.

Line 19 to 24 receives user inputs into the 2D array. Input data (exam scores) are stored into the 2D array which combines row number and column number to locate where the data should be stored.

The program creates a *two-by-two* 2D array when the user supplied 2 for number of **students** and 2 for number of **courses**. Using the inputs from the user (see figure 3-2.10), Table 3-2.2 shows a tabular representation of the 2D array. Each row represents a student and each column represents the exam **score** for each student in a particular course.

Table 3-2.2 The 2D array content after user input

	Column 0	Column 1
Row 0	80	85
Row 1	90	95

2.9 Summary

Two-dimensional arrays come in handy when you have to work with data which could be expressed in a tabular form. Populating or traversing the content of a 2D array is simple when you use nested for loops. The outer for statement controls the row and the inner for statement controls the column, therefore traversing a 2D array at any point requires a [row number] [column number] combination. The nested for statements traverses a 2D array row-wise; meaning, each 2D array traversal gets to the end of each row before processing another row.

ARRAYLIST in JAVA

Arraylist is based on the basic array data structure. It is widely accepted and used because it offers a number of flexible and easy-to-use functionalities. According to BeginnersBook (2017), in recent times, most developers choose Arraylist over array because it has proven to be a very good alternative to the traditional array.

Declaring ArrayList

To declare an ArrayList, you need to specify only two things; the name of the structure and data type the structure will contain. The syntax for creating an ArrayList is given as:

```
ArrayList<Data Type> nameOfArrayList = new ArrayList<Data Type>();
```

For example, to create an ArrayList of student names, you could do:

```
ArrayList<String> students = new ArrayList<String>();
```

This creates an ArrayList structure called **students** which stores any number of student names, because ArrayList is dynamic, you do not have to specify its length when declaring it.

program the
plied by the user
ed.

into the 2D array

ents and 2 for
s a tabular
the exam score

in 1

xpressed in a
ted for loops.
erefore

The nested for
df each row

fers a number
t times, most
he traditional

data type the

ees, because

Add Elements to ArrayList – add () method

ArrayList defines a predefined method called **add()** which is used to add elements to an ArrayList once it is created. The behaviour of this method is polymorphic; it can add elements at the end of the structure or insert elements at a specific location in a ArrayList.

To add element at the end of the structure, use this syntax:

```
nameOfArrayList.add("element");
```

For example, to add a student name at the end of the students ArrayList structure, you could do:

```
students.add("Abeiku French");
```

To add element at a specific index location in the ArrayList, use this syntax:

```
nameOfArrayList.add(index, "Element");
```

For example, to insert a name at a particular location in the student ArrayList, you could do:

```
students.add(5, "Abeiku French");
```

This adds the student's name Abeiku French at the fourth (4th) index position in the student ArrayList. Remember that ArrayList is based on the structure of array thus, it is zero (0) indexed.

Find Index Position of Element in ArrayList – indexOf() method

There are times you may want to insert elements before or after a particular element in the ArrayList, you could use this method to know the position to insert element before or after. The syntax of this method is given as:

```
int containerToReceiveIndexPosition = nameOfArrayList.indexOf ("element to find");
```

For example, to know the index position of "Abeiku French" in the students ArrayList, you could do:

```
int position = students.indexOf("Abeiku French");
```

This returns the index of element "Abeiku French" in the students ArrayList structure to the variable position.

Modify/Change Element in ArrayList – set() method

To modify or change an element in an ArrayList we have to indicate the index position of the element to be modified or changed and also provide the new element. This method will update the element in the ArrayList at the given index position with the provided new element. To modify or change any element in an ArrayList, use the syntax:

```
nameOfArrayList.set(index position, "new element");
```

For example, assume the element "Abeiku French" is the first item in the students ArrayList. To change the element "Abeiku French" in the students ArrayList structure, you could do:

```
Students.set(0, "Mame Aba");
```

This updates the element at index position 0 which stores item "Abelku French" to "Mame Aba";

Remove Element from ArrayList – remove() method

As in the case of the add() method, the remove() method is also polymorphic – it can remove element based on the name of the element; it can also remove element based on index position of element.

To remove element using its name, use the syntax:

```
ArrayListName.remove("element to remove");
```

For example, to remove the student "Mame Aba" from the students ArrayList, you could do:

```
Students.remove("Mame Aba");
```

This removes the student using the name of the student in the ArrayList.

To remove element based on index position, use this syntax:

```
ArrayListName.remove(index position of element to remove);
```

For example, to remove student from the students ArrayList based on index position of the element, you could do:

```
students.remove(0);
```

This removes student located at index position zero (0) of the students ArrayList.

Iterate through ArrayList – Foreach

In Java, the foreach loop is mostly preferred over the for-statement to iterate over arrays or Java Collections such as Arrays, ArrayLists, Lists, Set etc. The foreach is also known as enhanced loop because:

- It automatically detects starts and end of a Collection
- It can loop through heterogeneous items in a Collection
- It dynamically controls iteration; developers do not have to bother about any control variable
- It knows where a Collection ends hence, does not read beyond the last element etc.

The syntax of foreach loop is given as:

```
For(Type item : Collection){  
    //body of foreach loop  
}
```

Type: indicates the data type of the Collection from which element are going to iterated over.

Item: Stores returned element in the Collection duration iteration

Collection: This is the Java structure which store multiple data such as ArrayList, Array, etc.

ArrayList Example 1

In this example, we will demonstrate how to create, initialize, add and prints elements from an ArrayList object. Let's get started by following the steps below:

1. Create a class in the Collection project; name the class *StudentLists*
2. Type the following code into your class

```
1. package arrays;
2. import java.util.ArrayList;
3. import java.util.Iterator;
4. public class StudentLists {
5.
6.     public static void main (String [] args) {
7.         // create ArrayList of students
8.         ArrayList<String> students = new ArrayList<String>();
9.
10.        // Populate students object with 5 names
11.        students.add("Mame Aba");
12.        students.add("Mame Akua");
13.        students.add("Mansowah Fosua");
14.        students.add("Ewurabena Akyedze");
15.        students.add("Baaba Oforiwa");
16.
17.        // Display arraylist content using for loop
18.        System.out.println("Using for loop...");
19.        for (int i = 0; i < students.size(); i++) {
20.            System.out.print(students.get(i) + "\t");
21.        }
22.
23.        // display arraylist content using for each
24.        System.out.println();
25.        System.out.println();
26.        System.out.println("using for each...");
27.        for (String student : students) {
28.            System.out.print(student + "\t");
29.        }
30.    }
31. }
```

3. Run your program

4. Your output should be similar the output in figure 3-3.1

```

class FooBar {
    int foo;

    public FooBar(int bar) {
        this.foo = bar;
    }

    public String toString() {
        return "Foo: " + String.valueOf(foo);
    }
}

```

From the class FooBar, the toString() method implemented in figure 3-3.2 shows how to print FooBar Objects. We could create an instance of the FooBar class as shown in figure 3-3.3 and once the Object is created, we could now use the println() method to the the fooBar Object because the toString() method of the class indicated to the println() method how to print such Objects.

```

FooBar fooBar = new FooBar(15);

```

Figure 3-3.3 Instantiating an object of the Foo bar class

We will treat Reference data type (class) in a later chapter. We shall see how to combine the toString() method when we treat Classes.

Using for loop: The for loop is best suited for situations when you have a heterogeneous ArrayList- data type elements varies. Figure 3.3.4 shows how to use the for loop to print the students ArrayList.

```

for (int i = 0; i < students.size(); i++) {
    System.out.print(students.get(i) + "\t");
}

```

Figure 3-3.4 for statement to print array list content

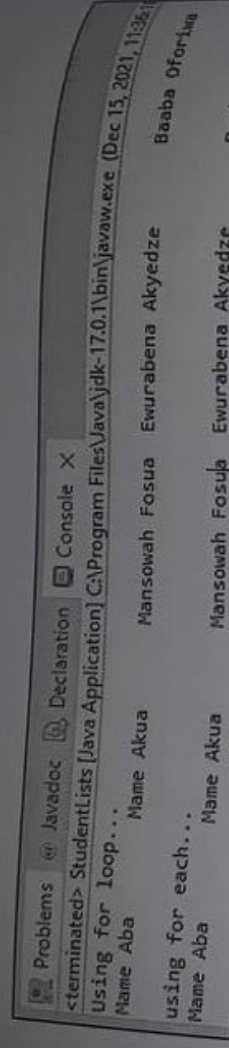
The code uses the get method of ArrayList to return the content of the ArrayList. The get method is to remove the square brackets and commas in the list of elements in an ArrayList see figure 3-3.1

Using foreach: Another option to print from a collection such as array (1D or 2D) or ArrayList or any kind of Java collection is by using the foreach looping construct. It is simple and easy to use. The StudentLists program implements a foreach loop using the code:

```

for (String student : students) {
    System.out.print(student + "\t");
}

```

ArrayList Example 1: Code Example

The `StudentsLists` class creates an `ArrayList` at line 9 and adds student names to the students `ArrayList` Object. The student Object is a `String ArrayList` which stores names of students.

Line 12 to 16 uses the `add()` method of the `ArrayList` class to add elements into the structure. Five students were added into the structure.

Line 20 to 22 is a for-statement construct which displays the content of the students `ArrayList`. There are many ways to print elements of an `ArrayList`, from line 28 to 30 implements a for each construct to print the content of the same students `ArrayList`.

Using `println` Method: The `println` method could be used directly only if the `ArrayList` is of a primitive data type. Primitive data types in Java include byte, short, integer, float, long, double and boolean. Strings are not primitive data type but due to the level of support Java gives it, it is safe to classify `String` under primitive data types.

The syntax for printing an `ArrayList` using the `println` method is given as:

`System.out.println(ArrayList object);`

Example is found at line 21 of the `StudentLists` class

Note that all primitive data types could be printed directly to the standard output device – the monitor. They do not have to be parsed into a printable format. Reference data types cannot be parsed by the `println` method directly, so the method cannot print Objects of Reference type.

Using `toString` Method: This method does not have the shortcomings of the `println` method. The reason non-primitive Objects cannot be printed to the output directly is that the compiler does not know how to print them. The `toString` method allows you to tell the compiler how to print Objects of a class to the standard output. Consider the example below:

```

class FooBar {
    int foo;

    public FooBar() {
        this.foo = 1;
    }

    public String toString() {
        return "foo: " + foo;
    }
}

```

From the class `FooBar`, we can create Objects. We could create as many Objects as we want. We could also create Objects of the class `FooBar` in the `main` method of the `FooBar` class.

Figure 3

Figure 3

We will treat `FooBar` as a class. We will treat `FooBar` as a class. We will treat `FooBar` as a class.

Using for loop
type elements

```

for (int i = 0; i < arr.length; i++) {
    // ...
}

```

Figure 4

The code uses `remove()` to remove the `Object` from the `ArrayList`.

Using `foreach`
of Java collection framework. The `foreach` loop is used to iterate over the elements of a collection. The `foreach` loop is used to iterate over the elements of a collection.

```

for (Object obj : arr) {
    // ...
}

```

ArrayList Example 2

We shall consider another example of a heterogeneous ArrayList which stores different data types. We shall remove, modify and delete elements from the structure. Let's complete the example by following the steps below:

1. Create a new class in the Collection project; name the class *MixedTypes*
2. Type the following code into your class

```

1. package arrays;
2. import java.math.BigDecimal;
3. import java.util.ArrayList;
4. import java.util.Collections;
5. import java.util.Iterator;
6. import java.util.Scanner;
7.
8. public class MixedTypes {
9.     public static void main(String[] args) {
10.         // Create heterogeneous ArrayList
11.         ArrayList<Object> stock = new ArrayList<Object>(0);
12.         Scanner scanner = new Scanner(System.in);
13.
14.         // initialize scanner to treat newlines as the separator, and use next() instead;
15.         // solves problems with nextLine()
16.         scanner.useDelimiter(System.lineSeparator());
17.
18.         System.out.println("How many input do you intend to make");
19.         int input = scanner.nextInt();
20.
21.         // for loop to accept input
22.         for (int i = 0; i < input; i++) {
23.             System.out.println("Name of product ");
24.             stock.add(scanner.next());
25.
26.             System.out.println("Quantity in stock: ");
27.             stock.add(scanner.nextInt());
28.
29.             System.out.println("Cost of product:");
30.             stock.add(scanner.nextBigDecimal());
31.         }
32.         // iterator to traverse ArrayList (another option)
33.         for (Iterator<Object> iterator = stock.iterator(); iterator.hasNext(); ) {
34.             Object object = (Object) iterator.next();
35.             System.out.println(object);
36.         }
37.         // Remove element - using index position
38.         stock.remove(0);
39.         // Reprinting elements to confirm removal

```

```

40. zero");
41. //
42. //
43. //
44. //
45. //
46. //
47. //
48. //
49. //
50. zero")
51. //
52. //
53. //
54. //
55. //
56. //
57. }
3. You c
4.

```

Fig
Array
We creat
ArrayList a
represent th
Using a for

Fig

Array

We creat
ArrayList a
represent th
Using a for


```

40. zero");
41.
42. //
43. for (Object element : stock) {
44.     System.out.println(element);
45. }
46. //
47. modify element at index position 1
48. stock.set(1, 1000);
49.
50. Reprinting elements to confirm removal
51. System.out.println("ArrayList after removing element @ index position
52. zero");
53.
54. foreach loop
55. for (Object element : stock) {
56.     System.out.println(element);
57. }

```

3. Run your program

4. You output should be similar to figure 3-3.6

```

1  How many input do you intend to make
2
3  Name of product
4  My Dream Ointment
5  Quantity in stock:
6  30
7  Cost of product:
8  23
9  My Dream Ointment
10 30
11 23
12 ArrayList after removing element @ index position zero
13 30
14 23
15 ArrayList after removing element @ index position zero
16 30
17 1000

```

Figure 3-3.6 Mixed types class output

ArrayList Example 2: Code Explanation

We created an ArrayList structure which could store data items of varying types – known as a heterogeneous ArrayList at line 11. Since the structure does not store one data type, we used a generic type – Object to represent the types of elements a user might input.

Using a for statement from line 22 to 31, the program receives into the ArrayList;

- First, String data representing name of item, at line 24
- Next, an int data representing quantity of items in stock, at line 27
- Then, finally, receive a decimal input representing the cost of the item, at line 30.

Line 33 to 36 demonstrate another mechanism to print the elements in the `ArrayList` this time, using the `Iterator` class. The `Iterator` class iterates through a `Collection` of items (also known as `List`). You have to specify the data type of the structure you want to iterate through with the help of the `Iterator` class thus, at line 33, the code `Iterator<Object> iterator` indicates to the `Iterator` to expect a `Collection` of elements of type `Object`. To check the data type of each `Object` in a `Collection`, you could use the `instanceof` operator. The syntax of this operator is given as:

nameOfObject instanceof Class;

The code in figure 3-3.7 uses the `instanceof` operator to check the data type of each element in the stock `ArrayList`.

```
for (int i = 0; i < stock.size(); i++) {
    if (stock.get(i) instanceof String)
        System.out.println(stock.get(i).toString() + " is a String type");
    else if (stock.get(i) instanceof Integer)
        System.out.println(stock.get(i) + " is Integer type");
    else if (stock.get(i) instanceof BigDecimal)
        System.out.println(stock.get(i) + " is a Big Decimal type");
}
```

Figure 3-3.7 Checks instance of each element in the array list

Still at line 33, in the second part of the for loop, the code `iterator = stock.iterator()` uses a method called `iterator()` to return an `iterator` over the elements in the `ArrayList`. The returned `iterator` is stored in the receiving `Object` called `iterator`. The last part of the for statement at line 33, is a code `iterator.hasNext()` which checks if there are more elements in the `ArrayList` to iterate (It returns true if there are elements in the `ArrayList` else, false).

The code `Object object = (Object) iterator.next()` at line 34 uses the `next()` method exposed by the `Iterator` class to return the elements in the `ArrayList` structure. The returned elements are stored in the container called `object`. The code at line 35 as you may recall, prints the element in the `object` container.

UNIT 4

EXCEPTION HANDLING

UNIT INTRODUCTION

Exceptions are unexpected errors (failures) that may occur during program execution. The solutions or interventions applied to remedy an unexpected failure in the smooth execution of a program is called handling the error.

Syntax of the try block is given as:


```
try {
    //Statements that may cause error in the program go here
}
```

While writing a program, enclose the areas of the program that might cause errors in the try block.
Syntax of the catch block is given as:

```
try {
    // Statements that may cause error in the program go here
}
catch (exception(type) e(object))
{
    //error handling code
}
```

Syntax of the finally block is given as:

```
try {
    // Statements that may cause error in the program go here
}
catch {
    //Handling exception
}
finally {
    //Statements to be executed
}
```

A finally block contains all the crucial statements that must be executed whether exception occurs or not.
The statements present in this block will always execute regardless of whether exception occurs in try block.

Nested Try-Catch

The syntax of nested try-catch is given as:

```
try {
    statement 1;
    statement 2;
    //try-catch block inside another try block
}
try {
    statement 3;
    statement 4;
    //try-catch block inside nested try block
}
try {
    statement 5;
    statement 6;
}
catch(Exception e2) {
```

```

18. //
19.     }
20.     //Exception Message
21.     }
22.     catch(Exception e1) {
23.         //Exception Message
24.     }
25.     }
26.     //Catch for main(parent/outer) try block
27.     catch(Exception e3) {
28.         //Exception Message
29.     }
30. }

```

CATCHING and HANDLING EXCEPTIONS

As we have seen in the previous section, exceptions in programs do occur so, as developers we must anticipate them in our code and find a way to handle them. In this section, we will consider some examples which illustrate how to handle exceptions in our code.

Try - catch Block Example 1

Let's consider an example where a division by zero occurs in a program, we shall use this erroneous code to explain how to use a try - catch block in Java. Follow the steps below to complete the program.

1. Create a new project in the Package Explorer; name the project *Exceptions*
2. Name the module for the project *unit4.exceptions*
3. In the *src* folder of the project, create a package; name the package *trycatch*
4. Create a class in the *trycatch* package; name it *DivideByZero*
5. Type the following code into your class. Remember to not repeat the class name

```

1. package trycatch;
2. public class DivideByZero {
3.
4.     public static void main(String args[]) {
5.         int num1, num2, result;
6.         try {
7.             // We anticipate that the code in this block can throw error.
8.
9.             num1 = 0;
10.            num2 = 62;
11.            result = num2 / num1;
12.
13.            System.out.println(result);
14.            System.out.println("This code is at the end of try block");
15.        }
16.        catch (ArithmeticException e) {
17.            // This block will only run if the anticipated error (Arithmetic exception)

```

Try - catch

In this section, we shall use this erroneous code to explain how to use a try - catch block in Java. Follow the steps below to complete the program.

1. Create a new project in the Package Explorer; name the project *Exceptions*
2. Name the module for the project *unit4.exceptions*
3. In the *src* folder of the project, create a package; name the package *trycatch*
4. Create a class in the *trycatch* package; name it *DivideByZero*
5. Type the following code into your class. Remember to not repeat the class name


```

18. // occurs in try block
19. System.out.println("An error has occurred, you cannot do a zero division, use a
    } different number");
20. catch (Exception e) {
21.     This is a generic Exception handler.
22.     System.out.println("Exception occurred");
23. }
24. System.out.println("I'm out of try-catch block in Java.");
25. }
26.
27. }

```

7. Run your program
8. Your output should be similar to the output in figure 4-2.1

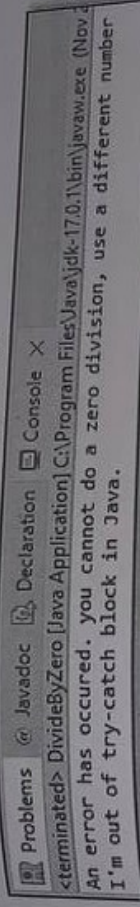


Figure 4-2.1 Divide by zero output

Try - catch Block Example 2

In this second example, we shall consider a program which receives numerical user inputs into an array and performs mathematical operations on the elements in the array. We shall use this example to demonstrate how multiple catch blocks could be associated to a single try block. Follow the steps below to complete the task.

1. Create a class in the Exceptions project; name the class `DivideByZero2`
2. Type you're the code below into your class. Remember not to repeat the class name.

```

1. package trycatch;
2. import java.util.Scanner;
3. public class DivideByZero2 {
4.     public static void main(String args[]) {
5.         try {
6.             int arr[] = new int[7];
7.             Scanner scanner = new Scanner(System.in);
8.             System.out.println("Input seven integer values. Press Enter after each input");
9.             for (int i = 0; i < arr.length; i++) {
10.                 arr[i] = scanner.nextInt();
11.             }
12.             arr[4] = 30/0;
13.             System.out.println("First print statement in try block");

```

```

14.         } catch(ArithmeticException e){
15.             System.out.println("Warning: ArithmeticException");
16.         }
17.     }
18.     catch(ArrayIndexOutOfBoundsException e){
19.         System.out.println("Warning: ArrayIndexOutOfBoundsException");
20.     }
21.     catch(Exception e){
22.         System.out.println("Warning: Some Other exception");
23.     }
24.     System.out.println("Out of try-catch block...");
25. }
26. }

```

3. Run your program. Input numeric values. Press Enter key after each input
4. Notice that the first catch block is what is executed.
5. Your output should be similar to the output of figure 4-2.2

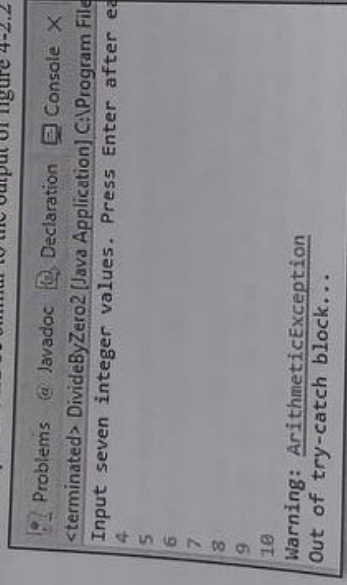


Figure 4-2.3 Divide by zero two class output

Try – catch – Finally Block Example

In this example, we will consider a program which reads a file from the file system to demonstrate how to use the finally block of a try – catch block. Follow the steps below to complete the program.

1. Create a new class in the Exception project; name the class *OpenFile*
2. Type the following code in your class

```

1. package trycatch;
2. import java.io.FileReader;
3. import java.io.IOException;
4.
5. public class OpenFile {
6.
7.     public static void main(String[] args) {

```

```

8.     try {
9.         read
10.        int i=
11.        while
12.        i=
13.        Sys
14.        }
15.    } catch
16.    } Sys
17.    } final
18.    } if(r
19.    t
20.
21.
22.
23.
24.
25.
26.    } Sys
27.    }
28.    }
29.
30. }

```

3. Run you

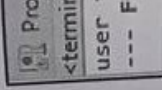


Figure 4-

Catching Mon

In Java SE 7, the exception. Th exception. Th

1. packa
2. impor
3. publ
- 4.
- 5.
- 6.
- 7.


```

8.      FileReader reader = null;
9.
10.     try {
11.         reader = new FileReader("aFile.txt");
12.         int i=0;
13.         while(i != -1){
14.             i = reader.read();
15.             System.out.println((char) i);
16.         }
17.     } catch (IOException e) {
18.         System.err.println("user file cannot be accessed");
19.     } finally {
20.         if(reader != null){
21.             try {
22.                 reader.close();
23.                 System.out.println("Read operation terminated here");
24.             } catch (IOException e) {
25.                 System.err.println("user file not found");
26.             }
27.         }
28.         System.out.println("--- File End ---");
29.     }
30. }

```

3. Run your application. Your output should be similar to the output in figure 4-2.4

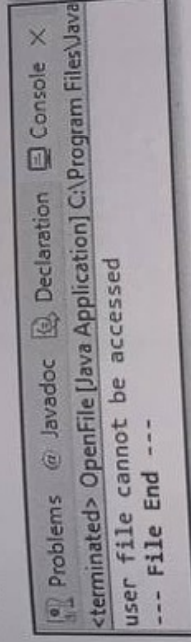


Figure 4-2.4 Finally block

Catching More Than One Type of Exception with One Exception Handler

In Java SE 7 and later, Java made it possible for a single catch block to handle more than one type of exception. This feature can reduce code duplication and lessen the temptation to catch an overly broad exception. This is also known as **multi catch**. Inspect the code below for how it is implemented.

```

1. package trycatch;
2. import java.util.Scanner;
3. public class DivideByZero2 {
4.     public static void main(String args[]){
5.         try{
6.             int arr[]=new int[3];
7.             Scanner scanner = new Scanner(System.in);

```

```

8.      System.out.println("Input seven integer values. Press Enter
9.      after each input");
10.     for (int i = 0; i < arr.length; i++) {
11.         arr[i] = scanner.nextInt();
12.     }
13.     arr[1]=30/0;
14.     System.out.println("First print statement in try block");
15. }
16. catch(ArithmeticException|ArrayIndexOutOfBoundsException|NumberFormatException
17. e){
18.     System.out.println("Warning: ArithmeticException");
19. }
20. catch(Exception e){
21.     System.out.println("Warning: Some Other exception");
22. }
23. finally {
24.     System.out.println("In the finally block....I'm always
25.     executing...");
26. }
27. }

```

Line 16 of the code uses a single catch block to handle multiple exceptions. Notice how the three exceptions in the multi catch block are separated by the pipe character |.

The output for the DivideByZero2 code above is shown in figure 4-2.5.

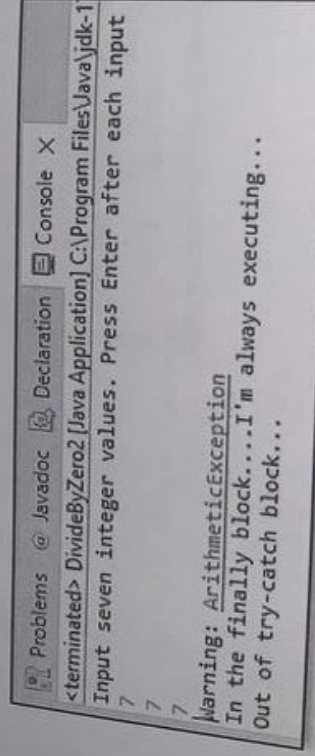


Figure 4-2.5 Multi catch block

The following are some useful Java exceptions:

1. **ArithmeticException**
It is thrown when an exceptional condition has occurred in an arithmetic operation.
2. **ArrayIndexOutOfBoundsException**
It is thrown to indicate that an array has been accessed with an illegal index. The index is either negative or greater than or equal to the size of the array.
3. **ClassNotFoundException**
This Exception is raised when we try to access a class whose definition is not found

4. **FileNotFoundException**
This Exception is thrown when the file is not found.
5. **IOException**
It is thrown when an I/O error occurs.
6. **InterruptedException**
It is thrown when a thread is interrupted.
7. **NoSuchFieldException**
It is thrown when a field does not exist in a class.
8. **NoSuchMethodException**
It is thrown when a method does not exist in a class.
9. **NullPointerException**
This exception is thrown when an attempt is made to use null as if it were an object.
10. **NumberFormatException**
This exception is thrown when the text is not a valid number.
11. **RuntimeException**
This represents a checked exception.
12. **StringIndexOutOfBoundsException**
It is thrown when the index is greater than the size of the string.

We will be looking at these exceptions in detail in the next chapter. Java is an Object-Oriented language. This unit will discuss the basic features and behaviour of Java.

The basic building blocks of Java are classes and objects. Examples of these are given below.

Java Class

Classes in Java are the basic building blocks of the language. They exhibit certain characteristics and behaviour, which are discussed in the next chapter.

Press Enter

block");

FormatException

);

'm always

three exceptions

FileNotFoundException

4. This exception is raised when a file is not accessible or does not open.

IOException

5. It is thrown when an input-output operation failed or interrupted

InterruptedException

6. It is thrown when a thread is waiting, sleeping, or doing some processing, and it is interrupted.

NoSuchFieldException

7. It is thrown when a class does not contain the field (or variable) specified

NoSuchMethodException

8. It is thrown when accessing a method which is not found.

NullPointerException

9. This exception is raised when referring to the members of a null object. Null represents nothing

NumberFormatException

10. This exception is raised when a method could not convert a string into a numeric format.

RuntimeException

11. This represents any exception which occurs during runtime.

StringIndexOutOfBoundsException

12. It is thrown by String class methods to indicate that an index is either negative or greater than the size of the string

UNIT 5

CLASSES AND OBJECTS

UNIT INTRODUCTION

We will be looking at how using Java, we can create our own object which could be used as data types. Java is an Object-Oriented programming language; the objects we shall create must belong to a defined class. This unit will demonstrate how to create classes together with their characteristics and behaviour in Java.

OBJECT and CLASSES

The basic building blocks in the Java programming language are classes, they define the characteristics, features and behaviour of an object. Classes are used to model real-world entities. We shall use varying examples to drum home the classes and objects concept.

Java Class

Classes in Java as already mentioned, mimic real-world entities. Real real-world objects have features and behaviour, developers simply model Java classes based on how real-world objects behave and the features they exhibit. Modelling is simply creating these features and behaviour into a Java class. After modelling a

real-world object into a class, we can, based on the class, create as many objects as we want. Each of these objects will possess the features and behaviour of the class.

Therefore, a Java class could be said to be blueprint which defines how an object must be created. Consider a real-world entity such as a Cat, it has some specific features and behaviour; a cat has a name, age, colour, etc.; it exhibits behaviour such as walking, running, eating, purring. We could model the entity cat into a Java class which exhibits the features and behaviour mentioned. Here is a sample code which models the Cat entity.

```

1. package ClassObject;
2. public class Cat {
3.     // fields/features of cat
4.     public String colour;
5.     public int age;
6.
7.     // Constructor of cat
8.     public Cat (String itsColour, int itsAge) {
9.         colour = itsColour;
10.        age = itsAge;
11.    }
12.
13.    // Behaviour of cat
14.    public void setColour (String newColour) {
15.        colour = newColour;
16.    }
17.
18.    public void setAge (int newAge) {
19.        age = newAge;
20.    }
21. }

```

The real-world cat entity, has been modelled into a Java class as seen in the code above, subsequent sections will give meaning to the code, so do not worry if you do not understand.

Declaring Classes

A class is defined or declared by following the following rules:

1. Name of a class must always begin with a capital letter
2. The body area of a class should be demarcated using opening and closing braces
3. Modifiers also known as access modifiers (**public**, **private**, etc.) may begin the name of the class
4. The constructor of a class must have same name as the class, must begin with a **public** modifier and must have no return type.

The name of the class is the identifier of the class, it is followed by the opening and closing braces, which defines the body of the class. The body of the class is where all the code which contains features and behaviour of the entity goes. Modifiers define the scope of the class; it describes which other classes could access it.

Constructors are necessary for every class since it is which that creates instances (objects) of the class. In creating the instance, we could define the state of each object through constructors. For example, in our

any objects as we want. Each of these
an object must be created. Consider
behaviour; a cat has a name, age, colour.
We could model the entity cat into a
is a sample code which models the

```
age) {
```

```
}) {
```

code above, subsequent
stand.

osing braces

y begin the name of the class
begin with a **public** modifier

ing and closing braces, which
h contains features and
es which other classes

es (objects) of the class. In
tors. For example, in our

Cat class, the constructor, while instantiating objects of the class sets the object's state with its name, its colour, and its age.

Member Variables

Member variables are also known as instant variables, instant variables are visible (accessible) to all methods and constructors of a class. There are three kinds of variables that could be created in a class:

- **Field:** is a member variable which is accessible by all other members of a class
- **Local variable:** is a variable which is declared in a method or a block of code within a class
- **Parameter:** is a variable declared with method declaration. They are found in the method head.

The Cat class declares two fields. A field declaration follows this order:

- **Modifier:** the default is public, but we could use private, protected, etc. to determine the accessibility of the field. A public field is accessible by any object that can access the class.
- **Field type:** it defines the type of data the field should store
- **Field's name:** it is the identifier of the field

The fields within the Cat class are:

```
public String colour;  
public int age;
```

The two fields are public, meaning they could be accessed by any object. The first and second fields are String fields, meaning they can only store String data. The field type for the second is int, which means the age field store in int data.

Method Definition

A method is a block of code to model behaviour of objects. Data could be passed to methods through parameters. By convention, the first (or only) name of a method should be a verb. A method could be defined by using the syntax:

```
<access modifier> return type methodName (<parameter listings separated by a comma>){  
    //body of method  
}
```

For example:

```
public int sum (int number1, int number2){  
    //body of sum method goes here  
}
```

- **Access modifier:** The access modifier of this method is public, which is optional, it could be private if we intend the method to be accessible only by the containing class.

- **Dynamic:** as we intend the method to accessible only by the containing class.

- **Dynamic:** as we intend the method to accessible only by the containing class.

be returned by the method. If a method does not return any value, use **void**.

Method Name: This is the identifier of the block of code. The block of code in the example is identifiable by the name `sum`. By convention, we use camel casing when naming methods.

name of a method should be a verb, if it's a multi-world name, the first word should be a verb, followed by adjectives, nouns, etc. Example: `getAge`, `setName`, `compareTo`, `isEmpty`.

Parameters: getAge, setName, compareTo, isEmpty.

in the parentheses after the method name, parameters are optional.

Method Body: This is where implementation of the behaviour of the entity goes including local variables.

The name of a method must be unique; name used by a method within a class cannot be assigned to a different method in the same class. This naming principle is not always true if two methods have the same name but one is overloaded.

The Java language differentiates between two methods based on their signatures. Method signatures make it possible for methods to have same name only if they have different parameters listing; the order in the listing is also crucial in distinguishing methods apart. The following examples use the same method names yet are different based on their signatures:

```
public int sum (int number1, int number2){
    //body of sum method goes here
}
```

```
public int sum (int number1, float number2){
    //body of sum method goes here
}
```

```
public int sum (float number1, int number2){
    //body of sum method goes here
}
```

Cat myJ
This code create
to the reference

Overloaded methods are thus differentiated by the number and type of data types of parameters passed to a method. Method overloading reduces code readability so you should not use them often in a class.

Constructors

Java is an Object-Oriented programming language, it sees everything as objects. These objects ought to be created out of a class which contains the features and behaviour of the object. So, when an object is created out of a class, it knows what its abilities and features are. We have also said that a class is a blueprint which defines how an object is created. The question you may be asking yourself is, how are these objects created? The answer is *constructors*. As the name implies a constructor creates (constructs) something, in this case, that something is called *object*. We have already seen a constructor in the Cat class. We shall be seeing more examples of constructors later in this unit.

The syntax for constructor is given as:

```
<access modifier> class name (<parameter listings>){  
    //body of constructor  
}
```

Observing the constructor syntax, you will realize that it looks quite similar to the method syntax except for some few differences. Note that a constructor must have the following mandatory components:

- access modifier: must always be public
- name of a constructor: must be the name of the class,
- return type: there must be no return type in constructor declaration

Optionally, a constructor could be used to initialize instance variables (fields) of a class. In that case, a constructor requires parameters which would be used for the instantiation.

In our Cat class example, the constructor was created in the class as:

```
public Cat (String String itsColour, int itsAge) {  
    colour = itsColour;  
    age = itsAge;  
}
```

- The return type for this class is public
- It has the same name as the class name (Cat)
- It has no return type
- It initializes the fields of the class with parameters

To create an object of this class called say, *myJoseph*, we call the constructor of the class by using the **new** keyword such as:

```
Cat myJoseph = new Cat ("grey and white", 5);
```

This code creates a new Cat object in memory which is grey and white and at age 5 and assigns this object to the reference called *myJoseph*.

A class may define as many constructors as possible with each having different implementation. We could also create an empty parameter constructor in a class such as:

```
public Cat () {
    colour = "Black and white";
    age = 3;
}
```

Subsequently, we can create a new object of the class class by invoking the new keyword on the empty constructor as we did before such as:

```
Cat yourJoseph = new Cat();
```

This creates an object in memory and assigns it to the reference called **yourJoseph**. The object has a black and white colour and is at age 3.

Passing Data to Methods and Constructors

Parameters are a good way of passing data to methods or constructors. It provides a way to make methods or constructors dynamic; they can interact with different objects with each object passing different arguments. Parameters describe number and data types expected by a method or a constructor for use in its body. Arguments are the actual values passed in place of parameters when a method or a constructor is invoked. Arguments passed into a method or a constructor must follow the order of type and number of parameters as defined in the invoked method.

The code below illustrates a method that computes monthly payments for a loan. The data (parameters) it needs to perform the computation are; loan amount, interest rate, duration of loan, and future value of the loan. Follow the steps below to complete the task.

1. Create a new project in the package explorer; call the project *Classes*
2. Click Finish. Name the module *util6.classes*. Click create
3. Create a package in the src folder; name the package *classes.objects*
4. In the package create a new class; name the class *Amortization*
5. Type the following code into your class. Remember not to repeat the class names

```
1. package classes.objects;
2. import java.math.BigDecimal;
3.
4. public class Amortization {
5.     // fields
6.     private String customerName;
7.     private String contactNumber;
8.
9.     // empty constructor
10.    public Amortization () {
11.
12.    }
13.
14.    // non-empty constructor
15.    public Amortization (String name, String number) {
16.        customerName = name;
17.        contactNumber = number;
18.    }
19.
20.    //
21.
22.
23.
24.
25.
26.
27.
28.
29.
30.
31.
32.
33.
34.
35.
36.
37.
38.
39.
40.
41.
42.
43.
44.
45.
46.
47.
48.
49. }
```

Passing Argument

When a method in its signature, method before i method in the m

1. Create
2. Type th

1. packag
2. publi
- 3.
- 4.
5. //


```

18. }
19.
20. // basic class methods (setters & getters)
21. public String getCustomerName() {
22.     return customerName;
23. }
24.
25. public void setCustomerName(String customerName) {
26.     this.customerName = customerName;
27. }
28.
29. public String getContactNumber() {
30.     return contactNumber;
31. }
32.
33. public void setContactNumber(String contactNumber) {
34.     this.contactNumber = contactNumber;
35. }
36.
37. // loan repayment method
38. public double computePayment(double principal, double rate, int
    duration) {
    the formula is given by:  $A = P(r(1+r)^n) / ((1+r)^n - 1)$ .
    Where P is principal or loan amount (e.g. 410,000.00)
    r is interest rate per year (e.g. 20% divided by 12 months)
    n is total number of payments or duration
    double r = rate/100;
    double intermediateVal1 = r * (Math.pow((1+r), duration));
    double intermediateVal2 = Math.pow((1+r), duration - 1);
    double monthlyPayment = (principal * (intermediateVal1)) /
    (intermediateVal2);
    return monthlyPayment;
39. }
40. //
41. //
42. //
43.
44.
45.
46.
47. }
48.
49. }

```

6. This class cannot be executed since there is no main method in it. It can only compile.

Passing Arguments

When a method is called to run, we say the method has been invoked. If a method A defines parameter(s) in its signature, then a method which invokes another method must provide data (argument(s)) to the invoked method before it can work. We demonstrate this in the code below, we will invoke the computePayment() method in the main method and pass arguments to it. Follow the steps below to complete the task.

1. Create a class in the Classes package; name it *Main*
2. Type the following code into your class. Do not repeat the class name
1. package classes.objects;
2. public class Main {
- 3.
4. public static void main(String[] args) {
5. Create object based on non-empty constructor

```

6.      Amortization loan = new Amortization("Judith Armah", "0200000000");
7.      //
8.      // invoke computePayment method through loan object
9.      double amount = loan.computePayment(10000.00, 21, 60);
10.     System.out.println("Monthly repayment amount: " + amount);
11. }

```

3. Run your application
4. Your output should be as in figure 6-1.1

```

<terminated> Main (1) [Java Application] C:\Program Files\Java\
Monthly repayment amount: 2541.0

```

Figure 6-1.1 Main class output

Objects

Java classes define states and behaviour of objects. Classes created in Java offer blueprint instructions to creating object of a class. The process of creating objects out of a class is termed **instantiating**. An instance of a class is the object created out of a class. For example, in the Main class, at line 6, the reference loan, stores the instance of an object which belongs to the Amortization class created with new keyword.

Since a class is only a blueprint, we could instantiate as many objects as we desire from a class, the caveat is that, as with variables, each object must have a unique name. Hence, we could create several Amortization objects in the Main class such as:

```

Amortization carLoan = new Amortization("Judith Armah", "0200000000");
Amortization homeLoan = new Amortization("Ama Ghana", "020001111");
Amortization housingLoan = new Amortization("Kofi Ansah", "020002222");

```

Each of these statements has three parts:

- **Declaration:** The code set in *italics* are variable declarations that associate a variable name with an object type - *Amortisation*.
- **Instantiation:** The new keyword is a Java operator that creates the object.
- **Initialization:** The new keyword is followed by a call to a constructor, which initializes the new object.

Since an object is an instance of a class, it also contains all the fields and behaviour of the class it belongs to. It is by this reason that we could safely invoke the computePayment method through the loan object as loan.computePayment(10000.00, 21, 60) at line 8. All *public* modifiers of a class are invocable through what is known in Java parlance as the dot (.) operator. To invoke a method, use the syntax:

```
object.methodName(<arguments>);
```

Object Declaration

In unit 1 section 4, we learned that to declare a variable, we needed to follow this syntax:

Data type variableName;

This creates a container in memory which is identifiable by the variableName you provide. It also tells the compiler that the variable will store data of the type specified during the declaration. With primitive types, the compiler appropriately knows the amount of memory space to reserve for the variable. In the same way you create a variable, we also create a reference variable.

To create a reference variable, follow the syntax:

ClassName reference;

For example:

```
Amortization carLoan;
```

Where Amortization is the name of a class and carLoan is the name of the reference variable. The amount of memory space allocation is undetermined until an object is created and assigned to the reference variable. The new keyword is that which creates a new object based on a constructor of a class then assigns it to a reference variable. Hence, the actual object of our Amortization class was created using the code:

```
Amortization carLoan = new Amortization("Judith Armah", "0200000000");
```

The part in bold (to the right of the assignment statement) is what actually creates the object based on the constructor **Amortization("Judith Armah", "0200000000")**, and the object is then assigned to the carLoan reference variable. We could have declared the reference variable separately then later assign it to an object of the same type such as:

```
Amortization carLoan;  
carLoan = new Amortization("Judith Armah", "0200000000");
```

RETURN TYPES and THIS KEYWORD

In this section, we demonstrate with examples the concept of return type and the *this* keyword. We show through the examples how they are used and when return types and this keyword are used in a Java program.

Return Value

In the sequence of code execution, an invoked method after it has finished executing its instructions, returns execution control back to the calling method. Consider this example;

```
1. package classes.objects;  
2. public class Main {  
3.  
4.     public static void main(String[] args) {  
5.         Create object based on non-empty constructor  
6.         Amortization loan = new Amortization("Judith Armah",  
7.         "0200000000");  
8.  
9.     }  
10. }
```

7. computePayment method through loan object

In unit 1 section 4, we learned that to declare a variable, we needed to follow this syntax:

Data type variableName;

This creates a container in memory which is identifiable by the `variableName` you provide. It also tells the compiler that the variable will store data of the type specified during the declaration. With primitive types, the compiler appropriately knows the amount of memory space to reserve for the variable. In the same way you create a variable, we also create a reference variable.

To create a reference variable, follow the syntax:

ClassName reference;

For example:

```
Amortization carLoan;
```

Where `Amortization` is the name of a class and `carLoan` is the name of the reference variable. The amount of memory space allocation is undetermined until an object is created and assigned to the reference variable. The new keyword is that which creates a new object based on a constructor of a class then assigns it to a reference variable. Hence, the actual object of our `Amortization` class was created using the code:

```
Amortization carLoan = new Amortization("Judith Armah", "0200000000");
```

The part in bold (to the right of the assignment statement) is what actually creates the object based on the constructor `Amortization("Judith Armah", "0200000000")`, and the object is then assigned to the `carLoan` reference variable. We could have declared the reference variable separately then later assign it to an object of the same type such as:

```
Amortization carLoan;  
  
carLoan = new Amortization("Judith Armah", "0200000000");
```

RETURN TYPES and THIS KEYWORD

In this section, we demonstrate with examples the concept of return type and the `this` keyword. We show through the examples how they are used and when return types and this keyword are used in a Java program.

Return Value

In the sequence of code execution, an invoked method after it has finished executing its instructions, returns execution control back to the calling method. Consider this example;

```
1. package classes.objects;  
2. public class Main {  
3.  
4.     public static void main(String[] args) {  
5.         Create object based on non-empty constructor  
6.         Amortization loan = new Amortization("Judith Armah",  
7.         "0200000000");  
8.     }  
9. }
```

7. invoke computePayment method through loan object
8. }


```

    }
    find maximum number in array
    public double findMax(double[] numbers) {
        double max = numbers[0];
        for (double number : numbers) {
            if (number > max) {
                max = number;
            }
        }
        return max;
    }

    public void printResult(double number) {
        System.out.printf("The maximum number is %s:", number);
    }
}

```

We need to create another class which has the main method in to execute our MaximumNumber class

3. Create another class in the Classes package; name the class **Main**

1. Type the following code into your Main class

```

1. package classes.objects;
2. public class Main {
3.
4.     public static void main(String[] args) {
5.
6.         create instance of MaximumNumber class
        MaximumNumber maxNumber = new MaximumNumber();
7.
8.         create a double array
        double[] readings = {47,89,78,56,98.8,80,78.34,85.65};
9.
10.        invoke methods simultaneously
        maxNumber.printResult(maxNumber.findMax(readings));
11.
12.    }
13.
14.
15. }

```

2. Run your program

3. Your output should be similar to figure 6-2.1

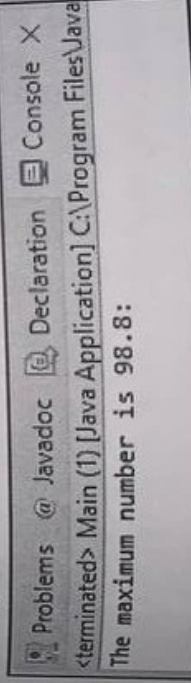


Figure 6-2.1 Maximum number class output

```

0, 21, 60);
t: " + amount);

```

okes the services of
main method, the
inishes running its
e main method. In

es not return with
ring the method's
be stated as void.
ment ought to be
de snippet below;

e of the keyword
seen in the body

e of value it will
uch a method, it
+ 8D) is double

value and one

The This Keyword

Within a method or a constructor, the *this* keyword is a reference to the current object – the object whose method or constructor is being called. You can refer to any member of the current class within a method or a constructor by using *this*. The most common reason for using the *this* keyword is because a field may be shadowed by a method or constructor parameter. Let's consider an example to explain the *this* keyword better.

This Keyword Example

For example, let's code a program that could find the minimum of three numbers. Follow the steps below to complete the task.

1. Create a new class in the Classes package; name the class *MinOf3*
2. Type the following code into your class

```
1. package classes.objects;
2. public class MinOf3 {
3.     // declare data members of the class
4.     private float num1;
5.     private float num2;
6.     private float num3;
7.
8.     // declare non-empty constructor
9.     public MinOf3 (float num1, float num2, float num3) {
10.         this.num1 = num1;
11.         this.num2 = num2;
12.         this.num3 = num3;
13.     }
14.
15.     // method to find minimum number
16.     public void findMin3() {
17.         if (this.num1 < this.num2 && this.num1 < this.num3) {
18.             System.out.println(num1 + " is the minimum of the three");
19.         } else if (this.num2 < this.num1 && this.num2 < this.num3) {
20.             System.out.println(num2 + " is the minimum of the three");
21.         } else
22.             System.out.println(num3 + " is the minimum of the three");
23.     }
24.
25. }
```

3. In the Main class, add the following code

```
MinOf3 min = new MinOf3(1, 0, 2);
min.findMin3();
```

3. Run your program.
4. Your output should be similar to figure 6-2.2

In unit 6, you s
Interfaces are i
shall as well d
class, i.e. how

Imagine a situ
Imagine a situ
from differen
Interface clas
of the groups
interface corr

Java Interface

Similar to a c
be instantiat
must contain
body), defat

Developers
main focus
you can the

Think of a
your exten
of the US
interface.
implement

object — the object whose
it class within a method or
is because a field may be
explain the this keyword

Follow the steps below

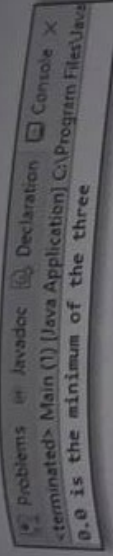


Figure 6-2.2 Min of 3 class output

UNIT 6

JAVA INTERFACES AND INHERITANCE

UNIT INTRODUCTION

In unit 6, you saw an implementation of the Interface class, in this unit we shall delve quite deep into what Interfaces are in Java, what they are used for, why and how to use Java Interfaces in your program. We shall as well discuss how you can use data members and other resources belonging to a class in another class, i.e. how a subclass can inherit fields and methods from a superclass.

JAVA INTERFACES

Imagine a situation where you have been given a group assignment to create a program using Java, or imagine a situation where you have won a contract to develop a program with other groups of programmers from different programming backgrounds. To create such programs, you are most likely to implement Interface classes; each group would be able to write their code without bothering or knowing how the rest of the groups or members of the group write theirs. Yet, when all is said and done, the program should interface correctly and work properly. This is where Java Interfaces come in handy.

Java Interfaces

Similar to a class, an *Interface* is only a reference type. Interfaces in the Java programming language cannot be instantiated, they could only be *implemented* or *extended* by other classes. As a rule, Java Interfaces must contain only constants, method signatures (abstract methods i.e. methods with header but with no body), default methods, static methods or nested types.

Developers use Interfaces to insist on what a class needs to do to be able to implement an interface. The main focus of Java Interfaces is to separate what you want a class to do from what the class actually does, you can then create a variety of classes which can variedly implement the same Interface.

Think of a **USB** port (otherwise called **USB interface**), using the same USB interface, you could plug in your external Hard Disc Drive, USB mouse, flash drive, printers, etc. To appreciate Java Interfaces, think of the USB pluggable devices as separate classes with different implementations for the same USB interface. Such is the idea behind Java interfaces; to create a structure which is independent of implementing classes and these classes have the liberty to implement the Interface how they want to.

```
{  
the three";  
num3) {  
the three";  
the three";  
}
```


Benefits of Interfaces

Java interfaces afford developers the following benefits:

- **Total abstraction:** An interface method only stores the method signature (abstract method) and not the method definition. This allows implementing classes to implement such methods how they want to.
- **Multiple inheritance:** Interfaces make it possible for developers to implement multiple inheritances as the conventional way of inheriting multiple parent classes may result in ambiguity.
- **Loose coupling:** Coupling refers to the degree of dependence of class on another class. Using Interfaces, you are able to de-couple a method's implementation from its definition (signature of a method).

Java Interface Syntax:

The structure of a Java Interface is similar to a Java class, the only difference is replacing the keyword **class** with **interface**. Syntax of Java Interface is therefore given as:

```
<modifier> interface InterfaceName {  
    //Declare Constant Fields;  
    //Declare Methods;  
    //Default Methods;  
}
```

Note that method signature in an interface does not include opening and closing curly braces, they only end with a semicolon.

The public access specifier indicates that the Interface could be accessible by any class within any package. If you choose not to define access specifier of the Interface, then the Interface will be accessible to classes in the same package with the Interface.

An Interface (technically called **extend**) can also use other interfaces. There is no capping on the number of other Interfaces an Interface could extend. The Interface declaration includes a comma-separated list of all the Interfaces that it extends as shown below.

```
public interface interfaces extends Interface1, Interface2, Interface3 {  
    // constant declarations  
    // base of natural logarithms  
    double E = 2.718282;  
    // method signatures  
    void doSomething (int i, double x);  
    int doSomethingElse(String s);  
}
```

Java Interface Example 1

We shall consider an example which implements Interface to calculates the area of geometrical shapes and for each shape, we will code a different method. All the methods in the class will be loosely coupled. Follow the steps below to complete the task.

1. Create a Java project in the package explorer; name the project *Interfaces*
2. Name the module *unit7.interfaces*
3. Create a package in the project; name the package *unit7.interfaces.inheritances*
4. In the *src* folder, create an Interface class. To do that, follow the steps below:
 - a. Right-click the *src* (source) folder, select *New*
 - b. Select *Interface* >> Name the interface *InterfaceArea*
 - c. Click *Finish*
5. Type the following code into your interface class

```
1. package unit7.interfaces.inheritances;
2. public interface InterfaceArea {
3.     //
4.     public void Square();
5.
6.     //
7.     public void Circle();
8.
9.     //
10.    public void Rectangle();
11.
12.    //
13.    public void Triangle();
14.
15.    //
16.    public void Parallelogram();
17.
18.    //
19.    public void Trapezoid();
20.
21. }
```

6. Create another class in the package to implement the Interface; name the class *AreaOfShapes*
7. Type the following code into your class exactly as shown below. Remember not to repeat class names.

```
1. package unit7.interfaces.inheritances;
2. import java.util.Scanner;
3. public class AreaOfShapes implements InterfaceArea{
4.
5.     Scanner sc = new Scanner(System.in);
6.
7.     @Override
8.     public void Square() {
9.         System.out.println("Type length of a square");
10.        double length = sc.nextDouble();
11.        System.out.printf("Area of the square %s", (length * length));
```



```

12.         System.out.println();
13.     }
14.
15.     @Override
16.     public void Circle() {
17.         System.out.println("Type radius of a circle");
18.         double radius = sc.nextDouble();
19.         System.out.printf("Area of the circle is %s", (Math.PI * (radius
20.
21.         System.out.println();
22.     }
23.
24.     @Override
25.     public void Rectangle() {
26.         System.out.println("Type the length of a rectangle");
27.         double Length = sc.nextDouble();
28.         System.out.println("Type the width of the rectangle");
29.         double Width = sc.nextDouble();
30.         System.out.printf("Area of the rectangle is %s", (Length *
31.         Width));
32.         System.out.println();
33.     }
34.
35.     @Override
36.     public void Triangle() {
37.         System.out.println("Type the base length of a triangle");
38.         double base = sc.nextDouble();
39.         System.out.println("Type the height of the triangle");
40.         double height = sc.nextDouble();
41.         System.out.printf("Area of the triangle is %s", (0.5 * (base *
42.         height)));
43.         System.out.println();
44.     }
45.
46.     @Override
47.     public void Parallelogram() {
48.         System.out.println("Type the length of base of a parallelogram");
49.         double base = sc.nextDouble();
50.         System.out.println("Type the height of the parallelogram");
51.         double height = sc.nextDouble();
52.         System.out.printf("Area of the circle is %s", base * height);
53.         System.out.println();
54.     }
55.
56.     @Override
57.     public void Trapezoid() {
58.         System.out.println("Type the base length of the first parallel
59.         side of a trapezoid");
60.         double base1 = sc.nextDouble();
61.         System.out.println("Type the base length of the second parallel
62.         side of the trapezoid");
63.         double base2 = sc.nextDouble();
64.         System.out.println("Type the height of the trapezoid");
65.         double height = sc.nextDouble();
66.         double Area = (((base1 + base2)/2) * height);

```

```

62.     }
63.
64.     }
65. }

```

Alternatively, you could use the `Shape` class and implement the `Shape` interface. The `Shape` interface will provide the `area` method. You can then implement the `area` method in the `Circle`, `Rectangle`, and `Trapezoid` classes. The `Shape` interface will provide the `area` method. You can then implement the `area` method in the `Circle`, `Rectangle`, and `Trapezoid` classes.

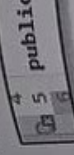


Figure 7-1.1 Problem

To implement the abstract

8. Click the problem
9. Now, you can

Note: When an instance

methods declared in

interface follow the

10. Create a main
11. Type the following

1. package un
- 2.

3. public class
4. public static

- 5.
6. Are

7. are
8. Sys

- 9.
10. are

11. Sys
- 12.

13. Sys
14. are

- 15.
16. Sys

17. are
- 18.

- 19.
20. Sys

21. are
- 22.

- 23.

```

62. System.out.printf("Area of the Trapezoid is %s",Area);
63. System.out.println();
64. }
65. }

```

Alternatively, you could make the eclipse IDE insert the abstract methods in the interface class into the `AreaOfShapes` class automatically. If a class implements an Interface, the implementing class must automatically implement all the methods in the Interface. Therefore, the implementing class must implement all unimplemented methods in the Interface. For example, on the same line of our class `AreaOfShapes` definition, you should see the problem marker (error marker) displaying to the left of the line numbers as shown in figure 7-1.1.

```

5 public class AreaOfShapes implements InterfaceArea{

```

Figure 7-1.1 Problem marker prompting to use abstract methods in interface

To implement the abstract methods in the Interface, follow the steps below;

8. Click the problem marker >> double – click *Add unimplemented methods*
9. Now, you can add the codes above to the inserted methods

Note: When an instantiable class implements an Interface, it provides a method body for each of the methods declared in the Interface. To instantiate the implementing class to execute the methods in the interface follow the steps below:

10. Create a main class in the package; name the class *Main*
11. Type the following code into the class.

```

1. package unit7.interfaces.inheritances;
2.
3. public class Main {
4.     public static void main(String[] args) {
5.
6.         AreaOfShapes area = new AreaOfShapes();
7.         area.Circle();
8.         System.out.println();
9.
10.        area.Square();
11.        System.out.println();
12.
13.        System.out.println();
14.        area.Rectangle();
15.
16.        System.out.println();
17.        area.Triangle();
18.
19.        System.out.println();
20.        area.Parallelogram();
21.
22.        System.out.println();
23.        area.Trapezoid();

```

```

circle");
is %s", (Math.PI * (radius
rectangle");
e rectangle");
is %s", (Length *

```

```

f a triangle");
triangle");
%s", (0.5 * (base *

```

```

f a parallelogram");
parallelogram");
base * height);

```

```

first parallel

```

```

second parallel

```

```

oid");

```



```

24.
25. }
26. }

```

12. Run your program
13. Type the inputs required for the program
14. Press the Enter key after each input
15. Your output should be similar to figure 7-1.2

```

12 Area of the circle is 452.3893421169302
11 Area of the square 121.0
6 Area of the rectangle is 30.0
10 Area of the triangle is 60.0
12 Area of the parallelogram is 96.0
8 Area of the Trapezoid is 138.0

```

Figure 7-1.2 Area of shapes instantiation output

Java Interface Example 1: Code Explanation

A Java Interface class cannot be instantiated to create an object of a class. Therefore, the program creates a concrete class called `AreaOfShapes` which is instantiable and implements **all** the methods of the Interface

in the `Main` class the instantiation of the concrete class happens at line 6. The new object of the class is referenced to the `area` variable is subsequently used to call the methods of the concrete class. There are a class that implements an Interface, you should include the `implements` keyword in the class declaration as done at line 3 of the `public class AreaOfShapes implements InterfaceArea` code. A class can implement more than one Interfaces, so the `implements` keyword is followed by a comma-separated list of interfaces implemented by the class. By convention, the `implements` clause follows the `class` clause, if there is one.

The whole idea of an Interface is that, a concrete class will implement the methods of an Interface class the way it deems fit. For example, we can create an Interface of methods which connect to a database, then an implementing class could use this method and define different means of connecting to different databases.

INHERITANCE

Objects in the real-world interact with each other, some objects are derived from other objects which exhibit features and behaviour of those objects they came out of. In Java, we can model such relationships using the concept of inheritance. In the Java language, classes can be inherited (derived) from other classes, hence inheriting fields and methods of those classes.

Inheritance (Is-A relationship) in Java

The concept of inheritance modelling could be defined as; a class that is derived from another class. A derived class is called a **subclass** or **extended class** or **child class** or **derived class**. The class from which another class is derived from is called a **superclass** or a **parent class** or a **base class**. Inheritance provides a mechanism through which a class can inherit the fields of another class. When a class is in a relationship with another class, i.e. when a child class extends a parent class, it inherits all non-private members (fields and methods and nested classes) of the parent class. Note that, the constructors of parent classes are not methods and are therefore uninherited by child classes.

A subclass inherits all the members (fields, methods, and nested classes) from its superclass. Constructors are not members, so they are not inherited by subclasses, but the constructor of the superclass could be invoked from the subclass.

If you are able to easily describe two classes using the phrase "is a/an" amongst the two classes, then those classes can be in a relationship. Inheritance defines "is a/an" relationship between a parent class and a child class. The words **extends** and **implements** are Java keywords which are used to describe relationships in

Java

Inheritance Example

We shall consider code a program which mimics a basic raw banking app for a rural bank to demonstrate inheritance concept. Note that the code in this program are by no means a substitute for a real banking app. Follow the steps below to complete the task.

1. Create a new class (*superclass*) in the Interfaces project; name the class *BankAccount*
2. Type the following code into your class

the program creates a
ods of the Interface


```

1. package unit7.interfaces.inheritances;
2. public class BankAccount {
3.     // the bank account has four fields
4.     private String accountName;
5.     private String accountNumber;
6.     private String branch;
7.     private double balance = Double.MIN_VALUE;
8.
9.     // The bank account has one constructor
10.    public BankAccount (String accName, String accNumber, String branch) {
11.        this.accountName = accName;
12.        this.accountNumber = accNumber;
13.        this.branch = branch;
14.    }
15.
16.    // the bank account has seven methods
17.    public String getAccountName() {
18.        return accountName;
19.    }
20.
21.    public void setAccountName(String accountName) {
22.        this.accountName = accountName;
23.    }
24.
25.    public String getAccountNumber() {
26.        return accountNumber;
27.    }
28.
29.    public String getBranch() {
30.        return branch;
31.    }
32.
33.    public double getBalance() {
34.        return balance;
35.    }
36.
37.    public void deposit(double amount) {
38.        balance += amount;
39.    }
40.
41.    public double withdraw(double amount) {
42.        if ((balance -= amount) >= 0) {
43.            return -1D;
44.        }
45.        else {
46.            balance -= amount;
47.            return amount;
48.        }
49.    }
50. }

```

3. Create another class (*subclass*) in the Interfaces project; name the class *MobileMoneyAccount*
4. Type the following code into the class. Note: add the words *extends BankAccount* to the class declaration as shown in the code at line 2.

```

1. package unit7.interfaces.inheritances;
2. public class MobileMoneyAccount extends BankAccount{
3.     // the mobile money account adds two additional fields
4.     private String networkProvider;
5.     private String mobilePhoneNumber;
6.
7.     // the mobile money account has one constructor
8.     public MobileMoneyAccount(String accName,
9.                               String accNumber,
10.                              String branch,
11.                              String phoneNumber) {
12.         super(accName, accNumber, branch);
13.         this.mobilePhoneNumber = phoneNumber;
14.     }
15.
16.     // the mobile money account class has four methods
17.     public String getNetworkProvider() {
18.         return networkProvider;
19.     }
20.
21.     public void setNetworkProvider(String networkProvider) {
22.         this.networkProvider = networkProvider;
23.     }
24.
25.     public String getMobilePhoneNumber() {
26.         return mobilePhoneNumber;
27.     }
28.
29.     public void setMobilePhoneNumber(String mobilePhoneNumber) {
30.         this.mobilePhoneNumber = mobilePhoneNumber;
31.     }
32.
33.
34. }

```

Things You can do in a Subclass

A subclass inherits all of the public and protected members of its superclass, no matter what package the subclass is in. If the subclass is in the same package as its superclass, it also inherits the package-private members of the superclass. You can use the inherited members as is, replace them, hide them, or supplement them with new members:

- The inherited fields can be used directly, just like any other fields.
- You can declare a field in the subclass with the same name as the one in the superclass, thus hiding it (not recommended).
- You can declare new fields in the subclass that are not in the superclass.
- The inherited methods can be used directly as they are.
- You can write a new instance method in the subclass that has the same signature as the one in the superclass, thus overriding it.
- You can write a new static method in the subclass that has the same signature as the one in the superclass, thus hiding it.

- You can declare new methods in the subclass that are not in the superclass.
- You can write a subclass constructor which invokes the constructor of the superclass, either implicitly or by using the keyword `super`.
- A subclass cannot inherit the private members of its parent class; they are rather accessible through public or protected methods of the superclass.
- A nested class has access to all the private data members and methods of its enclosing class.
- Therefore, a public or protected nested class inherited by a subclass has indirect access to all of the private members of the superclass.

Casting Object

We have so far established that an object is of the type of a class from which it was created (instantiated). So that if we code:

```
public MobileMoneyAccount momo = new MobileMoneyAccount();
```

then momo is of type `MobileMoneyAccount`.

We have also said that every Java class implicitly inherits from the default Java *Object* superclass (see figure 7-2.1) thus, hierarchically, we can annotate that *MobileMoneyAccount* descends from the *BankAccount* which also descended from the default *Object* superclass. Therefore, we can comfortably conclude that *MobileMoneyAccount* is a *BankAccount* and also an *Object* and it could be used wherever *BankAccount* and *Object* could be used.

If we code:

```
Object myObject = new MobileMoneyAccount();
```

Then *myObject* is of type `Object` and `MobileMoneyAccount` – this is implicit casting. To check the type of any object, you could use the code:

```
if (myObject instanceof MobileMoneyAccount) {
    System.out.println("YES");
}
```

If subsequently we initialize *myObject* to a reference such as:

```
MobileMoneyAccount momo = myObject;
```

The compile will flag a type mismatch error because it cannot implicitly convert *myObject* to *MobileMoneyAccount* type. We can solve this error by explicitly casting *myObject* to *MobileMoneyAccount* type such as:

```
MobileMoneyAccount momo = (MobileMoneyAccount) myObject;
```

Interface Methods

As we have discussed already in the previous unit, an Interface class may consist of different types of method; abstract or default methods. Both types of method could be inherited in the same way as we inherit

instance methods⁵. When an inherited Interface or superclass provide default methods of the same signature, the Java compiler follows two inheritance principles to resolve the conflicts. These principles are:

- *Instance methods are preferred over Interface default methods. Consider this example;*

```
1. public interface Zoo {
2.     default void whoAmI () {
3.         System.out.println("I'm a zoo");
4.     }
5. }
```

```
1. public interface Pet {
2.     default void whoAmI () {
3.         System.out.println("I'm a pet");
4.     }
5. }
```

```
1. public class Visitor {
2.     public void whoAmI () {
3.         System.out.println("I'm a visitor");
4.     }
5. }
```

```
1. public class Exploration extends Visitor implements pet, zoo {
2.     public static void main (String... args){
3.         Exploration explore = new Exploration();
4.         explore.whoAmI();
5.     }
6. }
```

Execution of explore.whoAmI() code will run the instance method in the whoAmI () in the Visitor class to display "I'm a visitor".

- *Methods that are already overridden by other candidates are ignored. Consider this example.*

```
1. public interface Pet {
2.     default void whoAmI () {
3.         System.out.println("I'm a pet");
4.     }
5. }
```

```
6. public interface Lion extends Pet {
7.     default void whoAmI () {
8.         System.out.println("I'm a lion and I'm someone's lovely pet");
9.     }
10. }
```

```
1. public interface Hyena extends Pet { }
```

⁵Instance method are methods which require an object of a class to be created before they could be invoked


```

7. public class Meerkat implements Lion, Hyena {
8.     public static void main (String... args){
9.         Meerkat meerkat = new Meerkat();
10.        explore.whoAmI();
11.    }
12. }

```

Execution of `meerkat.whoAmI()` code will run the `Lion` interface method to display "I'm a lion" and "I'm someone's lovely pet".

UNIT 7

SECTION 3

POLYMORPHISM

In this section, we shall discuss how to make an object behave in many different ways to meet varying requirements. We shall illustrate how subclasses can behave differently from their parent class yet share some functionality of the parent class.

What is Polymorphism?

It is one of the core concepts of Object – Oriented Programming where different classes can be used with the same interface and each of these classes can implement its own behaviour of the interface.

The word **poly** means **many** and **morph** also means **to change** or **to form**, so you can also explain polymorphism as ability for a program to present the same interface for varying underlying forms.

We have been implementing this concept silently throughout this book. Consider the Integer data type for example, it is implicitly polymorphic since you can add, subtract, multiply and etc.

Polymorphism Example

To demonstrate polymorphic features of Java, follow the steps below to complete the task.

1. Alter the *BankAccount* class to the code below (We have included a *toString* method)

```

1. package unit7.interfaces.inheritances;
2. public class BankAccount {
3.
4.     // the bank account has four fields
5.     private String accountName;
6.     private String accountNumber;
7.     private String branch;
8.     private double balance = Double.MIN_VALUE;
9.
10. // The bank account has one constructor

```

```

11. public BankAccount (String accName, String accNumber, String branch) {
12.     this.accountName = accName;
13.     this.accountNumber = accNumber;
14.     this.branch = branch;
15. }
16. //
17. // the bank account has seven methods
18. public String getAccountName() {
19.     return accountName;
20. }
21. //
22. // public void setAccountName(String accountName) {
23. //     this.accountName = accountName;
24. // }
25. //
26. // public String getAccountNumber() {
27. //     return accountNumber;
28. // }
29. //
30. // public String getBranch() {
31. //     return branch;
32. // }
33. //
34. // public double getBalance() {
35. //     return balance;
36. // }
37. //
38. // public void deposit(double amount) {
39. //     balance += amount;
40. // }
41. //
42. // public double withdraw(double amount) {
43. //     if ((balance -= amount) == 0) {
44. //         return -1D;
45. //     } else {
46. //         balance -= amount;
47. //         return amount;
48. //     }
49. // }
50. // @Override
51. // public String toString() {
52. //     return "BankAccount [accountName=" + accountName + ",
53. //         accountNumber=" + accountNumber + ", branch=" + branch
54. //         + ", balance=" + balance + "]" ;
55. }

```

2. Alter the *MobileMoneyAccount* class to the code below (We have included a *toString* method)

1. package unit7.interfaces.inheritances;
- 2.
3. public class MobileMoneyAccount extends BankAccount{
- 4.
5. // the mobile money account adds two additional fields


```

6.     private String networkProvider;
7.     private String mobilePhoneNumber;
8.
9.     // the mobile money account has one constructor
10.    public MobileMoneyAccount(String accName,
11.                               String accNumber,
12.                               String branch,
13.                               String phoneNumber) {
14.        super(accName, accNumber, branch);
15.        this.mobilePhoneNumber = phoneNumber;
16.    }
17.
18.    // the mobile money account class has four methods
19.    public String getNetworkProvider() {
20.        return networkProvider;
21.    }
22.
23.    public void setNetworkProvider(String networkProvider) {
24.        this.networkProvider = networkProvider;
25.    }
26.
27.    public String getMobilePhoneNumber() {
28.        return mobilePhoneNumber;
29.    }
30.
31.    public void setMobilePhoneNumber(String mobilePhoneNumber) {
32.        this.mobilePhoneNumber = mobilePhoneNumber;
33.    }
34.
35.    public static void main(String[] args) {
36.        Object obj = new MobileMoneyAccount("S", "H", "B", "C");
37.
38.        //MobileMoneyAccount momo = (MobileMoneyAccount)obj;
39.        if (obj instanceof Object) {
40.            System.out.println("YES");
41.        }
42.    }
43.
44.    @Override
45.    public String toString() {
46.        return "MobileMoneyAccount [networkProvider=" + networkProvider +
47.               ", mobilePhoneNumber=" + mobilePhoneNumber
48.               + "]\n";
49.    }
50. }

```

3. Create a new class in the Interfaces project; name the class *InvestmentAccount*.
4. Type the following code into the class

```

1. package unit7.interfaces.inheritances;
2. public class InvestmentAccount extends BankAccount{
3.     // investment amount
4.     private double investment;

```

```

5. // duration indicates number of months to invest money
6. // private int duration;
7.
8. // rate indicates the rate of returns on investment
9. // private float rate;
10.
11. public InvestmentAccount(String accName, String accNumber, String
12.     branch, int duration) {
13.     super(accName, accNumber, branch);
14.     this.duration = duration;
15. }
16.
17. public int getDuration() {
18.     return duration;
19. }
20.
21. public void setDuration(int duration) {
22.     this.duration = duration;
23. }
24.
25. public float getRate() {
26.     return rate;
27. }
28.
29. public void setRate(float rate) {
30.     this.rate = rate;
31. }
32.
33. public double getInvestment() {
34.     return investment;
35. }
36.
37. public void setInvestment(double investment) {
38.     this.investment = investment;
39. }
40.
41. public double calRate(float rate) {
42.     investment += (rate/100) * investment;
43.     return investment;
44. }
45.
46. @Override
47. public String toString() {
48.     return "InvestmentAccount [investment=" + investment + ",
49.         duration=" + duration + ", rate=" + rate + "]\n";
50. }
51. }

```

5. Create another class in the Interfaces project; name the class *TestBankAccounts*

6. Type the following codes into the class

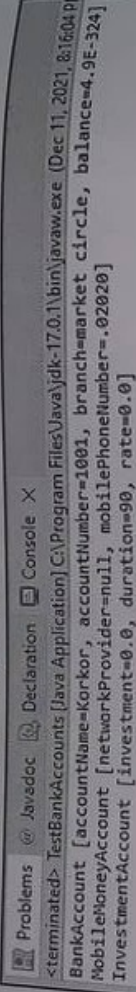
1. package unit7.interfaces.inheritances;


```

2. public class TestBankAccounts {
3.     public static void main(String[] args) {
4.         BankAccount account1, account2, account3;
5.
6.         account1 = new BankAccount("Korkor", "1001", "market circle");
7.         account2 = new MobileMoneyAccount("Sherifa", "1002", "beach
            road", "02020");
8.         account3 = new InvestmentAccount("Baaba", "1003", "Effiakuma",
            90);
9.
10.        System.out.println(account1.toString());
11.        System.out.println(account2.toString());
12.        System.out.println(account3.toString());
13.    }
14. }

```

7. Run your program
8. Your output should be similar to figure 7-1.1



```

<terminated> TestBankAccounts [Java Application] C:\Program Files\Java\jdk-17.0.1\bin\javaw.exe (Dec 11, 2021, 8:16:04 PM)
BankAccount [accountName=Korkor, accountNumber=1001, branch=market circle, balance=4.9E-324]
MobileMoneyAccount [networkProvider=null, mobilePhoneNumber=02020]
InvestmentAccount [investment=0.0, duration=90, rate=0.0]

```

Figure 7-1.1 Polymorphism

Polymorphism Example: Code Explanation

The code in the `TestBankAccounts` explains how polymorphism in Java works, at line 4, we create three reference variables `account1`, `account2` and `account3` of type `BankAccount`.

Line 7 to 9 create different objects each to a reference variable, for example, line 8 creates a `MobileMoneyAccount` object to the `account2` reference variable.

UNIT 1 SESSION 1

MY FIRST JAVA PROGRAM

We shall begin to put what we have discussed so far into practice in this section. We will create our first program in Java and get ourselves acquainted with the Eclipse IDE to understand the basic parts of any Java program. Subsequently in this section, we will explore examples regarding variables and data conversion.

First Java Program

To create any program in Java, you need two basic tools; an IDE to write your Java source codes and a JDK to compile your codes into bytecodes. The IDE could be any of the popular IDEs listed in Section 1 of Unit 1. If you have not installed JDK and the Eclipse IDE, visit Section 1 of Unit 1 to do so.

Our first program will follow the convention of first programs by beginners; a program to display a simple message – Hello world on the screen. Let's get started. Follow the steps below to create the program.

1. Open the Eclipse IDE
2. Click the Launch button if the Workspace path is correct.
Note: If you check the option **Use this as the default and do not ask again**, Eclipse will know where to store its configurations, thus, the Workspace window will not display in the next launch.
3. If it is the first time you are using the Eclipse IDE after installation, you will be required to select a Perspective¹ from the Welcome screen. Click Create a new Java Eclipse Project as shown in figure 1-4.1

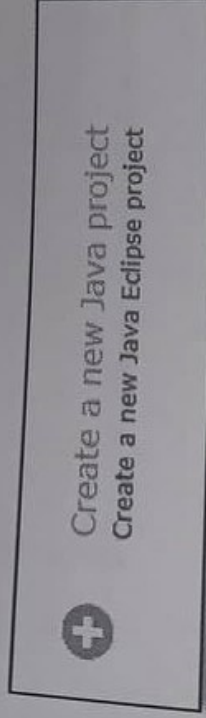


Figure 1-4.1 Java Perspective

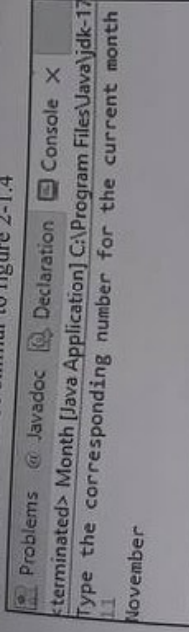
- Note:** You can change your Perspective later by selecting *Welcome* from the *Help* menu in the IDE.
4. If it is the first project you are creating after installing the Eclipse IDE, you will be required to name your Java project. In this example you will name your project *FirstJavaProject* as shown in the figure 1-4.2


```

32.         break;
33.     case 12: nameOfMonth = "December";
34.         Break;
35.     default: nameOfMonth = "Invalid month";
36.         break;
37.     }
38.     System.out.println(nameOfMonth);
39. }
40.
41. }
42.
43.

```

3. Run your application. Input the current month number. Press the Enter key
4. Your output should be similar to figure 2-1.4



REPETITION (ITERATION)

The While Loop and the Do-while loop

The syntax for the while loop could be expressed as:

```

while (condition) {
    //body of the loop: statements to execution as long as the condition remains true.
}

```

The While Loop Example

We consider an example which uses the while loop block to print numbers between 1 and 10. Follow the steps below to complete the task.

1. Create a new class in the decision project; name the class *OneToTen*
2. Type the following code into your class

```

1. package ifDecision;
2. public class OneToTen {
3.
4.     public static void main(String[] args) {
5.
6.         int count = 1;
7.         while (count < 11) {
8.             System.out.println("Count is: " + count);

```

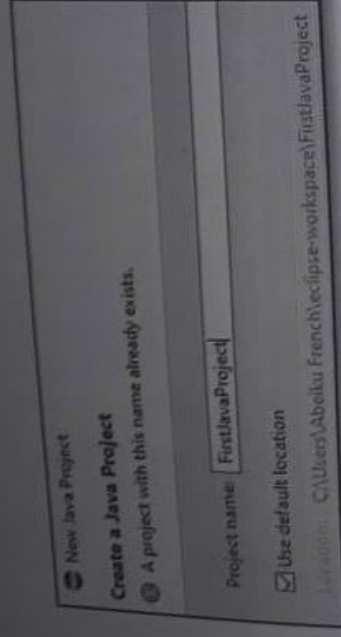


Figure 1-4.2 Java Project

5. Click the Finish button to move to the next window
6. After creating a project in the Eclipse IDE, you will be required to also create a module². In this example, we will use *hello.firstjavaproject* as the module name.
7. Click the Create button to create the Java project including module-info.java source file as shown in figure 1-4.3

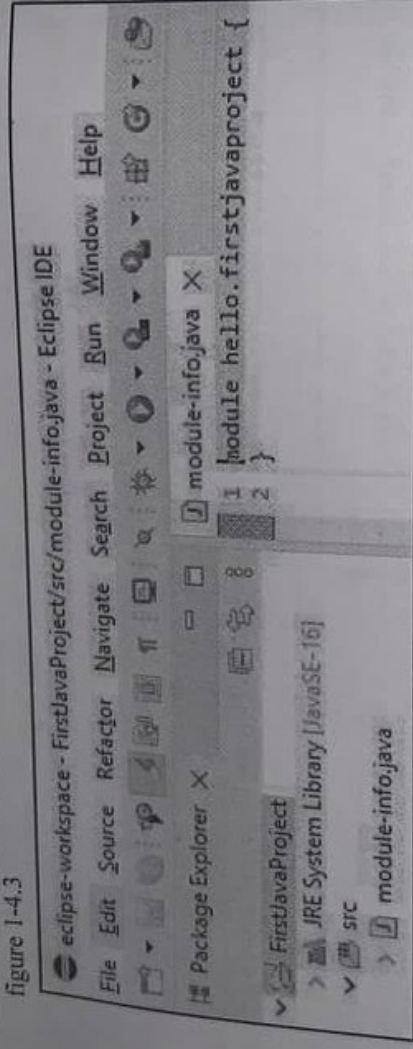


Figure 1-4.3 Module-info.java file

8. Next, you will create a package in your project. A package is used by developers to organize their codes. To create a package in your *FirstJavaProject*, follow the steps below:
 - a. Click the down arrow just before your project name in the package explorer of the Eclipse IDE to expose the project's content if not displaying already.
 - b. Right-click the src (source) folder >> select New >> select Package
 - c. Name the Package as *hello.firstjavaproject* >> Click the Finish button to create the package

Next, you are going to create a class in the package created in the project. The class will be called *Main*.
Note: You will notice that the name of the class ends with extension (.java) after the class is created, it was added automatically by the IDE.

9. To create the class, right-click the *hello.firstjavaproject* package in the project >> select New >> select Class. Name the class *Main* at where the cursor is blinking.

² A module is a named set of Java packages, resources, and native libraries. Module name begins with small letter.


```
count += 1;
```

```
    }
10. }
11. }
12. }
```

Figure 2-2.1 One to ten class

3. Run your application

Your output should be similar to figure 2-2.1

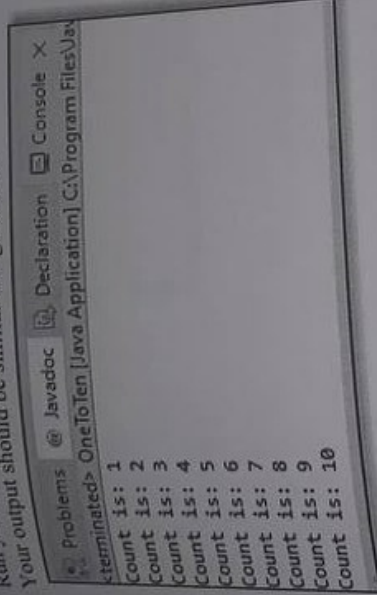


Figure 2-2.1 One to ten class output

The Do-while Loop

The syntax for the do-while loop could be expressed as:

```
do {
    //body of the loop: statements to execute while the condition is true
} while (condition);
```

The Do-while Loop Example

We will again create a program to print 1 to 10 but will use the do-while loop this time. Follow the steps to complete the code.

1. Create a new class in the decision project; name it *OneTen*
2. Type the following code into your class

```
1. package ifDecision;
2.
3. public class OneTen {
4.     public static void main(String[] args){
5.         int count = 1;
6.         do {
7.             System.out.println("Count is: " + count);
8.             count++;
9.         } while (count < 11);
10. }
```

10. You could make the IDE insert a *main* method into the class by checking the *public static void main* (*String [] args*) check box option (see figure 1-4.4).

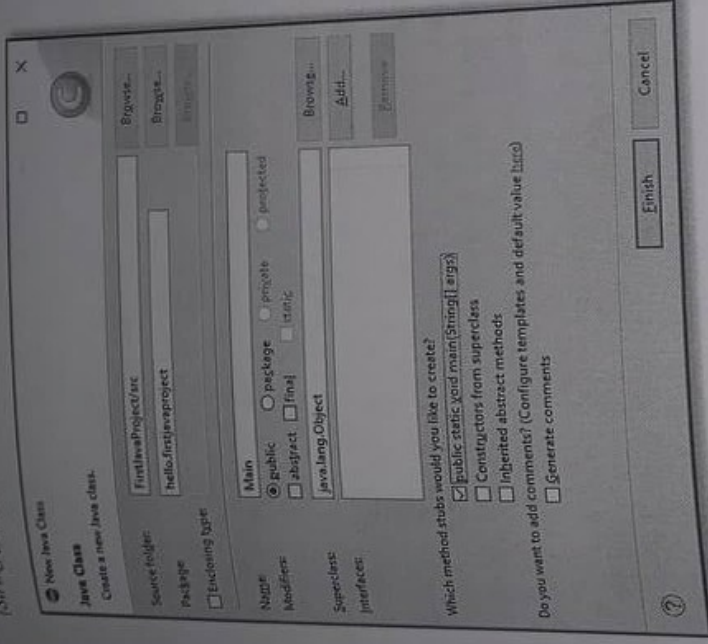


Figure 1-4.4 Main class configuration

11. Click the Finish button to create the Main class as shown in figure 1-4.5.

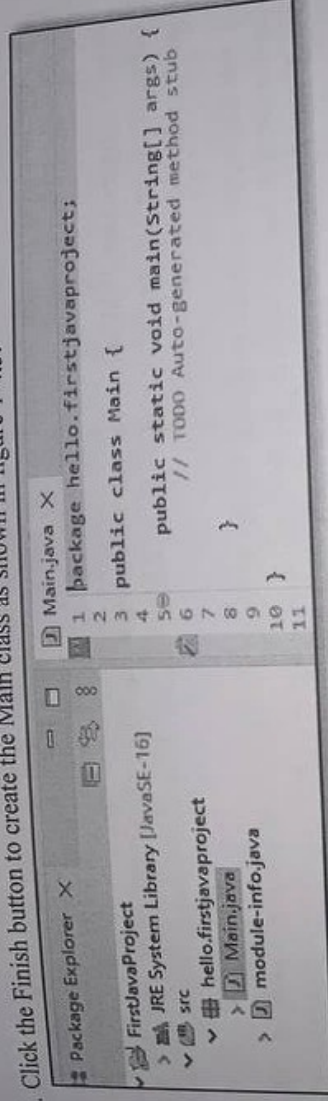


Figure 1-4.5 Main class with main method

12. We are now set to add our source code which will display the message Hello World. Remove the text in line six in the main method and type code as shown in figure 1-4.6

11. }

3. Run your program
4. Your output should be similar to the output in figure 2-2.2

```

Count is: 2
Count is: 3
Count is: 4
Count is: 5
Count is: 6
Count is: 7
Count is: 8
Count is: 9
Count is: 10
  
```

The for Statement

The syntax for the for statement could be expressed as:

```

for (initialization; conditional expression; increment/decrement)
{
    //body of the loop: statements to execute while the condition is true
}
  
```

- *Initialization:* this section of the loop is used to initialize the variable which indicates the starting point of the loop. An already declared variable could be used or a variable could be declared in this part which becomes a local variable to the for loop only. This portion of the construct executes only once.
- *Conditional expression:* this section evaluates the boolean condition to determine the entry and exit points of the loop; when true, the body of the loop is executed; when false; the execution flow control jumps the end of the for statement
- *Body of the loop:* Once the boolean condition evaluates to true, the statements in the body of the loop body are executed.
- *Increment/Decrement:* This section is used for updating the variable of the loop for the next iteration to be evaluated.
- *Loop termination:* Once the conditional expression evaluates to false, the loop terminates marking the end of the looping cycle.

The for-Statement Example

Let's see how to use the for loop to print the first ten counting numbers. Follow the steps below to complete the task.

1. Create a class in the decision project; name the class *ForOneToTen*
2. Type the following code into your class
 1. package ifDecision;

```

2. public class
3. public
4. {
5.     }
6.     }
7.     }
8.     }
9. }
  
```

3. Run your a
4. Your output

```

Count is: 1
Count is: 1
Count is: 1
Count is: 1
Count is: 1
Count is: 1
Count is: 1
Count is: 1
Count is: 1
Count is: 1
  
```

Figure 2-2.2

Branching state
statements allow
languages uses

- Break
- Continue
- Return

The break Statement

Using the break
expression and

The break statement
switch statement
look at how to

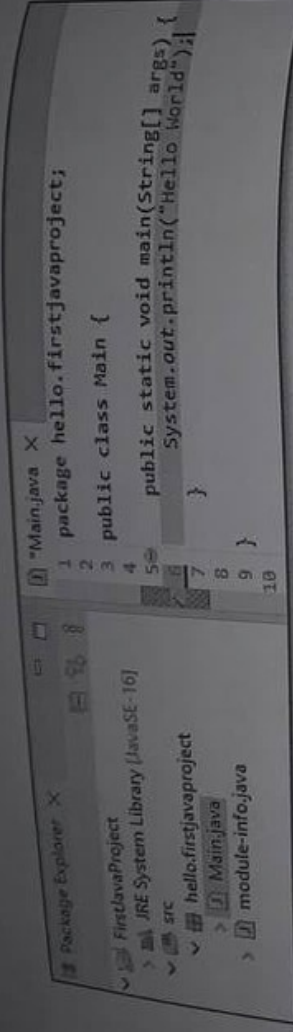


Figure 1-4.6 Code to display message

13. Right-click **Main.java** in the Package Explorer >> select **Run As** >> select **Java Application**. Your application should execute and display an output similar to figure 1-4.7.

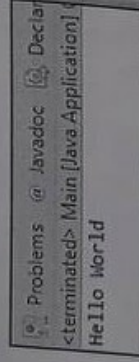


Figure 1-4.7 Output message

Note: If your program does not run, it may be due to error(s) in your code. Revisit figure 1-4.6 to make corrections.

JVM Configuration

There may be moments where the Eclipse IDE does not run your application not because there are errors in your code, but because the Java Virtual Machine (JVM) is not properly configured to run such applications. We are going to demonstrate to you how to configure your JVM so that it runs your programs.

Note: Configure your JVM only when you have correctly installed your JDK and have tested it at the command prompt with the DOS command `javac`. Refer to *How to Setup Environment Variables in Java in unit 1 section 1* on how to test your JDK.

In your project, you created a module in the `module-info.java` file and you also created a `Main` class in a separate file. You need to configure the JVM so that the `Main` class will include (or point to) the module path. Follow the steps below to configure your JVM to point the `Main` class to the path of the module.

1. Right-click the project *FirstJavaProject* in the Package Explorer and select **Properties**
2. In the **Properties** window, click **Run/Debug Settings** >> click the **New...** button.
3. Select **Java Application** in the **Select Configuration Type** window >> click **OK** to open the **Edit Launch Configuration properties** window
4. In the **Main class** field in empty, click the **Search ...** button to open **Select Main Type** window


```

2. public class ForOneToTen {
3.     public static void main(String[] args){
4.         for(int i=1; i<11; i++){
5.             System.out.println("Count is: " + i);
6.         }
7.     }
8. }
9.

```

3. Run your application

4. Your output should be similar to figure 2-2.7

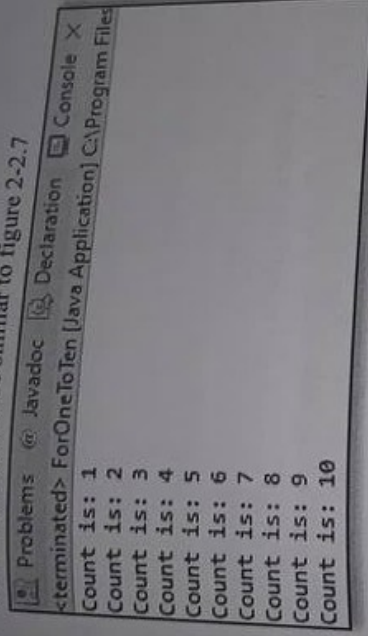


Figure 2-2.3 For one to ten class output

BRANCHING

Branching statements allow execution flow control to jump to a different part of the program. Branching statements allow us to exit from a control statement when a certain condition is met. Java programming languages uses three branching mechanisms, they are as follows:

- Break statement
- Continue statement
- Return statement

The break Statement

Using the break statement, we can force immediate termination of a loop, bypassing the conditional expression and any remaining code in the body of the loop.

The break statement has two forms: **labeled** and **unlabeled**. We used the unlabeled form when we discussed switch statement. You could also use an *unlabeled* break to terminate a for, while, or do-while loop. Let's look at how to use unlabeled break statement in the next example.

5. The Main class of your project should be listed in the Select Main Type window together with the package the Main class belongs to as shown in figure 1-4.8.
6. Select the Main class in the Matching items list box as shown figure 1-4.8
7. Click the OK button

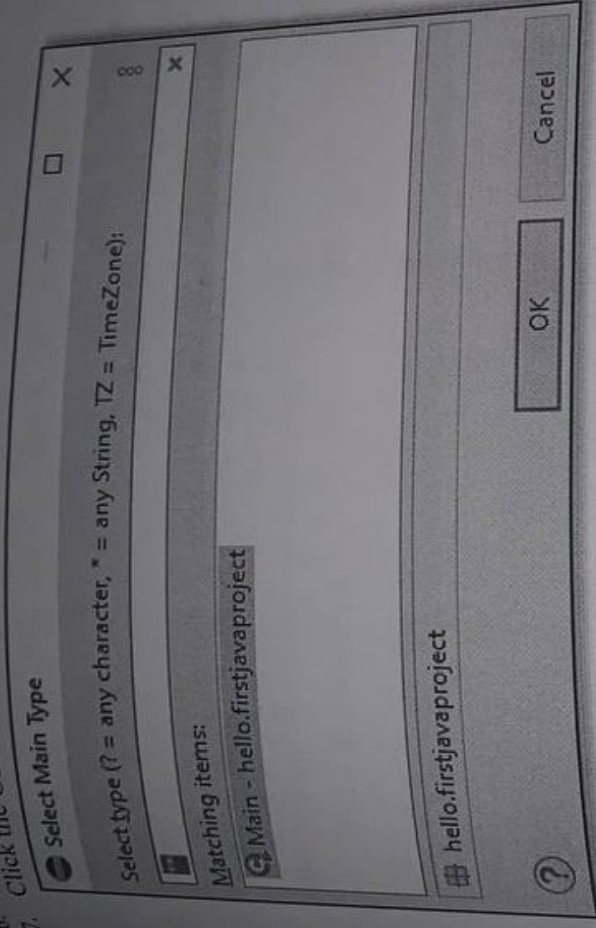


Figure 1-4.8 Virtual machine configuration

8. Click the Arguments tab in the Edit Configuration window
9. Specify the configuration arguments in the *VM arguments* field as shown in figure 1-4.9

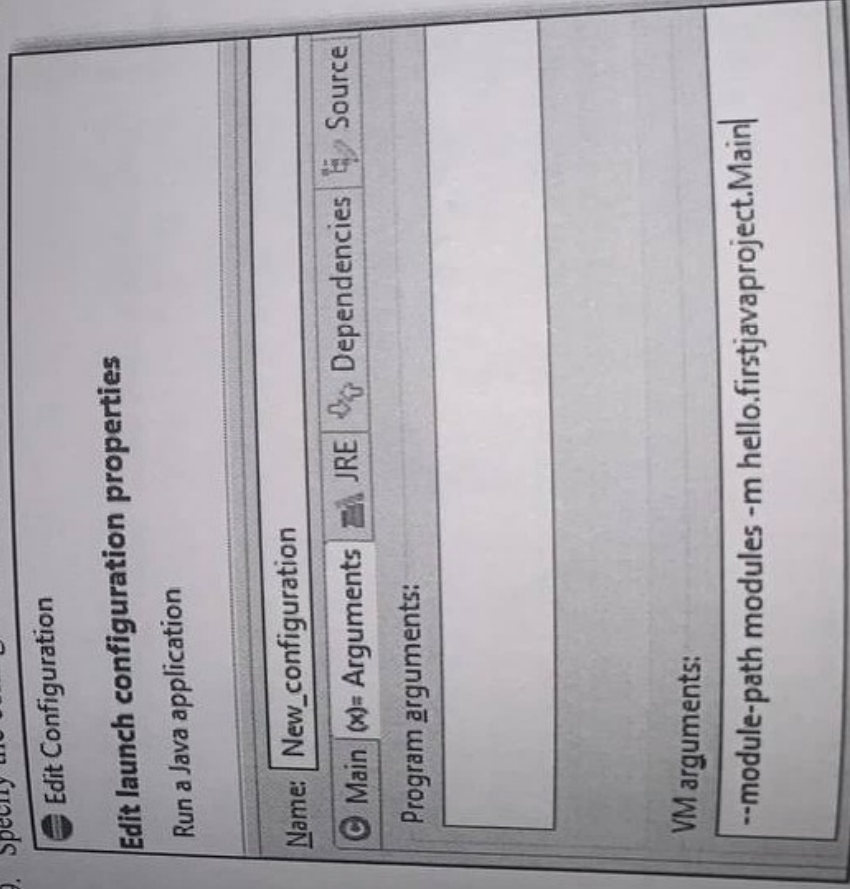


Figure 1-4.9 Virtual machine configuration arguments

The break Statement Example (Unlabeled)

We shall create a program which displays the first even number from a set of predetermined numbers. Once the first even number is found, execution flow must break. Follow the steps below to complete the task.

1. Create a class in the decision project; name the class *FirstEvenNumber*
2. Type the following code into your class

```
1. package ifDecision;
2.
3. import java.util.Iterator;
4. import java.util.Scanner;
5.
6. public class FirstEvenNumber {
7.
8.     public static void main(String[] args) {
9.
10.         int [] numbers = {1,3,5,4,8};
11.
12.         for (int i = 0; i < numbers.length; i++) {
13.             if (numbers[i] % 2 == 0) {
14.                 System.out.println(numbers[i]);
15.                 break;
16.             }
17.         }
18.     }
19.
20. }
21. }
```

3. Run your program.

4. Your output should be similar to figure 2-3.1

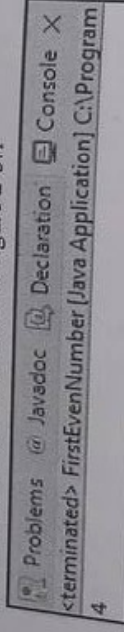


Figure 2-3.1 First even number class output

The break Statement Example (Labeled)

An *unlabeled* break statement terminates the innermost of *switch*, *for*, *while*, or *do-while* statement, but a labeled break terminates an outer statement.

We shall consider an example similar to the previous program, but this time we shall use nested for loops to search for two values in a predetermined set of numbers. The program will first search for even numbers, then search for an odd number in the given set of numbers contained in the array. When the values are found, a labeled break terminates the outer for loop (labeled "search"): Follow the steps below to complete the task.

Create
1. Type
pack
2. pub
3. pub
4.
5.
6.
7.
8.
9.
10.
11.
12.
13.
14.
15.
16.
17.
18.
19.
20.
21.
22.
23.
24. }

3. Ru
4. Y

The con

The con
skips to
In simpl
of the lo

The con

Let's cr
"p". If t
next ch

5. The Main class of your project should be listed in the Select Main Type window together with the package the Main class belongs to as shown in figure 1-4.8.
6. Select the Main class in the Matching items list box as shown figure 1-4.8.
7. Click the OK button

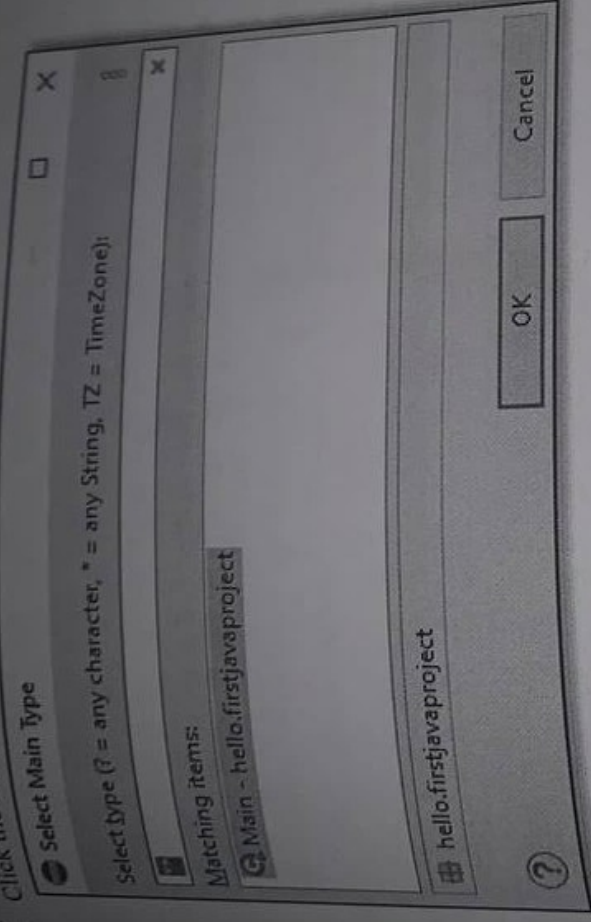


Figure 1-4.8 Virtual machine configuration

8. Click the Arguments tab in the **Edit Configuration** window
9. Specify the configuration arguments in the *VM arguments* field as shown in figure 1-4.9

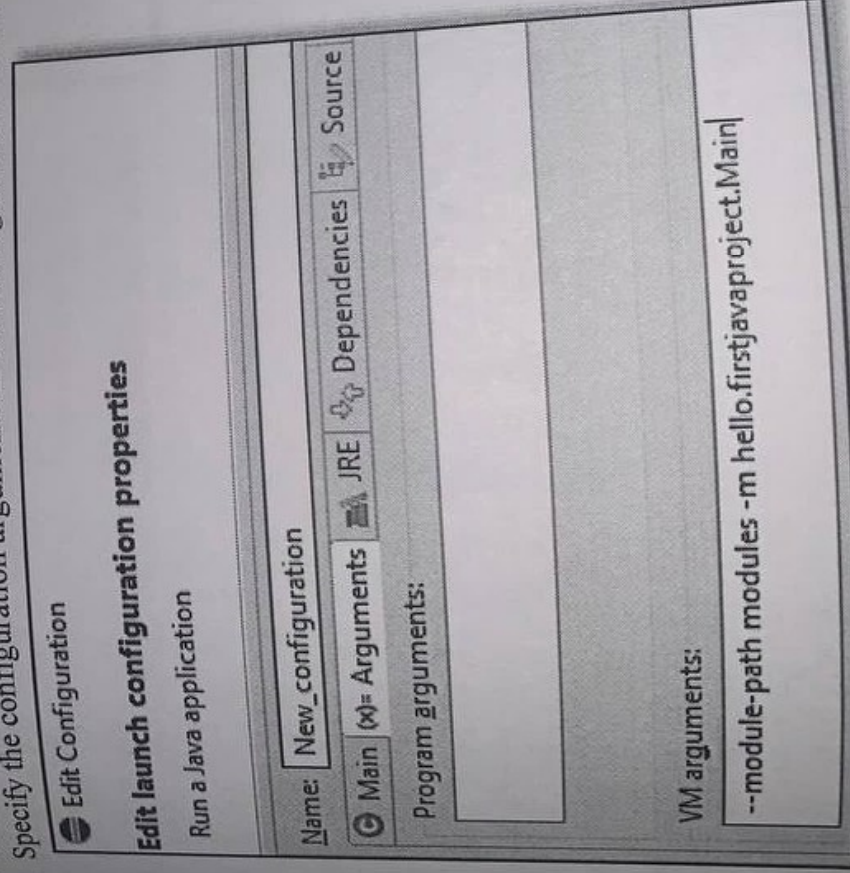


Figure 1-4.9 Virtual machine configuration arguments

Create a program in the decision project; name it *FirstEvenOddNumbers*

```

1. Type the following code into your class
2. package 1stDecision;
3. public class FirstEvenOddNumbers {
4.     public static void main(String[] args) {
5.         int [] numbers = {1,3,7,6,8,11,12,13};
6.         int even = 0, odd = 0;
7.
8.         search:
9.         for (int i = 0; i < numbers.length; i++) {
10.             if (numbers[i] % 2 == 0) {
11.                 even = numbers[i];
12.                 for (int j = 0; j < numbers.length; j++) {
13.                     if (numbers[j] % 2 == 1) {
14.                         odd = numbers[j];
15.                         break search;
16.                     }
17.                 }
18.             }
19.         }
20.         System.out.println("First even number is: " + even);
21.         System.out.println("First odd number is: " + odd);
22.     }
23. }
24. }

```

3. Run your application
4. Your output should be similar to figure 2-3.2

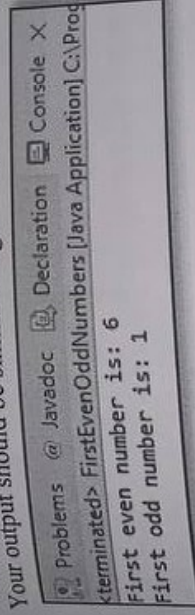


Figure 2-3.2 First even odd numbers class output

The continue Statement

The continue statement skips the current iteration of a for, while, or do-while loop. The *unlabeled* form skips to the end of the innermost loop's body and evaluates the boolean expression that controls the loop. In simple terms, the continue statement is used to skip a part of a loop and continues with the next iteration of the loop. It can be used in combination with for and while statements.

The continue Statement Example (Unlabeled)

Let's create a program which passes through a String, searching and counting every occurrence of the letter "p". If the current character is not a p, the *continue* statement skips the rest of the loop and proceeds to the next character. If it is a "p", the program increments the letter count. Follow the steps to complete the task.

10. Click the OK button >> Click Apply and Close button in the Properties window

Running Java Module program

We shall run the Java module application based on the configurations made in 4.3. The following steps shows how to run the application.

1. Right-click the Main.java class in the *Package Explorer*
2. Select *Run As >> Java Application*
3. The Main class will be listed in the Select Java Application window. Select the Main class as shown in figure 1-4.10

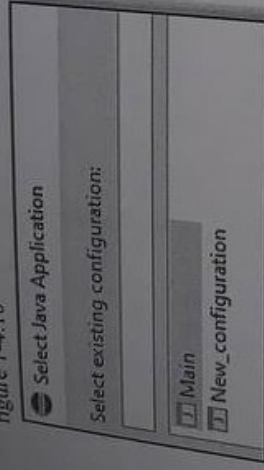


Figure 1-4.10 Run As configuration

4. Click the OK button
5. Your application runs and displays the message on the console as shown in figure 1-4.11

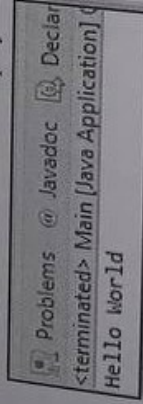


Figure 1-4.11 Output message

UNIT 2

CONTROL FLOW STATEMENTS (CONDITIONAL STATEMENTS)

UNIT INTRODUCTION

So far in Unit1, all our programs have been linear in form; they execute from the top of the code to the bottom of the code as they appear in the IDE; the compiler executes each line in sequence. However, in typical Java programs, execution flow is non-linear; implementation of conditional logical decisions makes execution flow either loop sometimes or branches to other portions of the code.

1. Create a class in the decision project; name the class *SearchCount*
2. Type the following code into your class

```

1. package ifDecision;
2.
3. public class SearchCount {
4.     public static void main(String[] args) {
5.
6.         String searchP = "peter piper picked a peck of pickled peppers";
7.         int len = searchP.length();
8.         int numberOfPs = 0;
9.
10.        for (int i = 0; i < len; i++) {
11.            // interested only in Ps
12.            if (searchP.charAt(i) != 'p') {
13.                continue;
14.            }
15.            // increment count of p's
16.            numberOfPs++;
17.        }
18.        System.out.println("Found " + numberOfPs + " Ps in the string.");
19.
20.    }
21. }

```

3. Run your program
4. Your output should be similar to that shown in figure 2-3.3

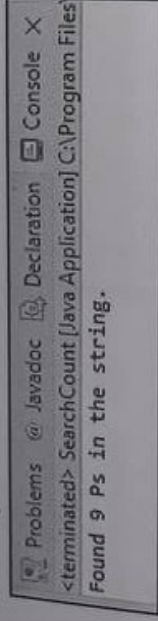


Figure 2-3.3 Search found class output

The continue Statement (Labeled)

A labeled continue statement skips the current iteration of an outer loop marked with the given label. A label for a loop could be used with a continue statement to specify where the control of execution is to be transferred.

The continue Statement Example (Labeled)

Let's consider a Java program which uses a nested for loop to print numbers. We will consider two codes; the first code uses labels, the second code does not. Follow the steps below to complete the task.

First code:

1. Create a class in the decision project; name the class *Numbers*